Lecture 6: Generative Adversarial Networks

Deep Generative Models

Sajjad Amini

Department of Electrical Engineering
Sharif University of Technology

Contents

1. Intuition
2. Foundation
3. Global Optimality
4. Training Algorithm
5. Deep Convolutional GAN
6. Mode Collapse
7. f-GAN
8. Bidirectional Generative Adversarial Networks
9. CycleGAN
10. Conclusion

Section 1

Intuition

Intuition

Likelihood vs Sample Quality [1]

Average log-likelihood and visual fidelity of samples are largely independent of each other when the data is high-dimensional.

Great Samples and Poor Log-Likelihood [1]

Assume your generative model has learned the following distribution:

$$p_\theta(\boldsymbol{x}) = \frac{1}{N}\sum_{i=1}^{N}\mathcal{N}(\boldsymbol{x}|\boldsymbol{x}_i, \epsilon^2\boldsymbol{I})$$

where ϵ is a small value. Then:

- **Great samples:** Because the model regenerates the training data with small noise.
- **Poor Likelihood:** Because the model likelihood over validation or test samples is low since they are disjoint from the training set.

Intuition

Great Log-likelihood and Poor Samples [1]

Assume your generative model has learned the following distribution:

$$p_\theta(\boldsymbol{x}) = 0.01 p_{\text{data}}(\boldsymbol{x}) + 0.99 \mathcal{N}(\boldsymbol{x}|\mathbf{0}, \boldsymbol{I})$$

- **Poor Samples:** because if we generate 100 samples, we expect the model to generate samples from $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ in almost 99 generations and generate a sample from $p_{\text{data}}(\boldsymbol{x})$ in 1 generation.
- **Great Likelihood:** needs some prerequisits!

Lower-Bounding Model Likelihood [1]

Assume the following model:

$$p_\theta(\boldsymbol{x}) = 0.01 p_{\text{data}}(\boldsymbol{x}) + 0.99 \mathcal{N}(\boldsymbol{x}|\mathbf{0}, \boldsymbol{I})$$

Then:

$$\begin{aligned}
\log p_\theta(\boldsymbol{x}) &= \log(0.01 p_{\text{data}}(\boldsymbol{x}) + 0.99 \mathcal{N}(\boldsymbol{x}|\mathbf{0}, \boldsymbol{I})) \\
&\geq \log(0.01 p_{\text{data}}(\boldsymbol{x})) \\
&= \log p_{\text{data}}(\boldsymbol{x}) - \underbrace{\log(100)}_{4.61}
\end{aligned}$$

where the inequality is due to the fact that log is monotonically increasing. Thus:

$$\mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})} [\log p_\theta(\boldsymbol{x})] \geq \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})} [\log p_{\text{data}}(\boldsymbol{x})] - 4.61$$

Intuition

Upper-Bounding Model Likelihood [1]

Assume an arbitrary model, from the KL divergence properties, we know:

$$\mathrm{KL}(p_{\text{data}}(\boldsymbol{x}) \| p_\theta(\boldsymbol{x})) \geq 0$$
$$\Rightarrow \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})} [\log p_{\text{data}}(\boldsymbol{x})] \geq \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})} [\log p_\theta(\boldsymbol{x})]$$

Intuition

Sandwiching Model Likelihood [1]

Assume the following model:

$$p_\theta(\boldsymbol{x}) = 0.01 p_{\text{data}}(\boldsymbol{x}) + 0.99 \mathcal{N}(\boldsymbol{x}|\mathbf{0}, \boldsymbol{I})$$

Then:

$$\begin{cases} \text{ML} \triangleq \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})} \left[\log p_\theta(\boldsymbol{x})\right] \\ \text{DL} \triangleq \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})} \left[\log p_{\text{data}}(\boldsymbol{x})\right] \end{cases} \Rightarrow \text{DL} - 4.61 \leq \text{ML} \leq \text{DL}$$

Intuition

Normal Distribution [2]

Given unit-variance uncorrelated normally distributed data with D dimensions, the log-likelihood of the data will be:

$$\frac{D}{2}\left(1+\ln(2\pi)\right)$$

Independent and Identically Distributed

Assume we have a random vector $\mathbb{X} \in \mathbb{R}^D$ with independently and identically distributed elements, then:

$$p(\boldsymbol{\mu}) = p([\mu,\ldots,\mu]) = \prod_{i=1}^{D} p(\mu) = \left(p(\mu)\right)^D$$

$$\Rightarrow \log p(\boldsymbol{\mu}) = D \log p(\mu)$$

From CIFAR10 to ImageNet

Consider the following garbage model (in terms of generating samples):

$$p_\theta(\boldsymbol{x}) = 0.01 p_{\text{data}}(\boldsymbol{x}) + 0.99 \mathcal{N}(\boldsymbol{x}|\mathbf{0}, \boldsymbol{I})$$

Then:

- Assume you are training a model over CIFAR10 dataset ($\boldsymbol{x} \in \mathbb{R}^{32\times32\times3}$) and data log-likelihood is ~ -1000. Then the garbage model likelihood is:

$$-1004.61 \leq \text{ML} \leq -1000$$

- Now for ImageNet dataset ($\boldsymbol{x} \in \mathbb{R}^{224\times224\times3}$), the likelihood is ~ -49000. Then the garbage model likelihood is:

$$-49004.61 \leq \text{ML} \leq -49000$$

Thus for the garbage distribution $p_\theta(\boldsymbol{x})$, we have a great likelihood while very poor samples.

Intuition

Altogether

Consider the following model distribution:

$$p_\theta(\boldsymbol{x}) = 0.01 p_{\text{data}}(\boldsymbol{x}) + 0.99 \mathcal{N}(\boldsymbol{x}|\mathbf{0}, \boldsymbol{I})$$

It has **Poor Samples** and **Great Likelihood**.

Likelihood Proficiency

Based on our discussion, likelihood may not be a good indicator of sample quality. Thus we need to explore more for a better distance (divergence) metric to fit the

Section 2

Foundation

Likelihood Free Divergence

Divergence Metric

In this section, we want to design a new divergence metric such that:

- Data distribution is the minimizer of divergence:

$$p_{\text{data}} = \underset{p}{\operatorname{argmin}}\, \mathcal{D}(p_{\text{data}}, p)$$

- The distance can be evaluated using samples of $p_{\text{data}}(\mathbb{X})$ and $p_\theta(\mathbb{X})$ distributions.

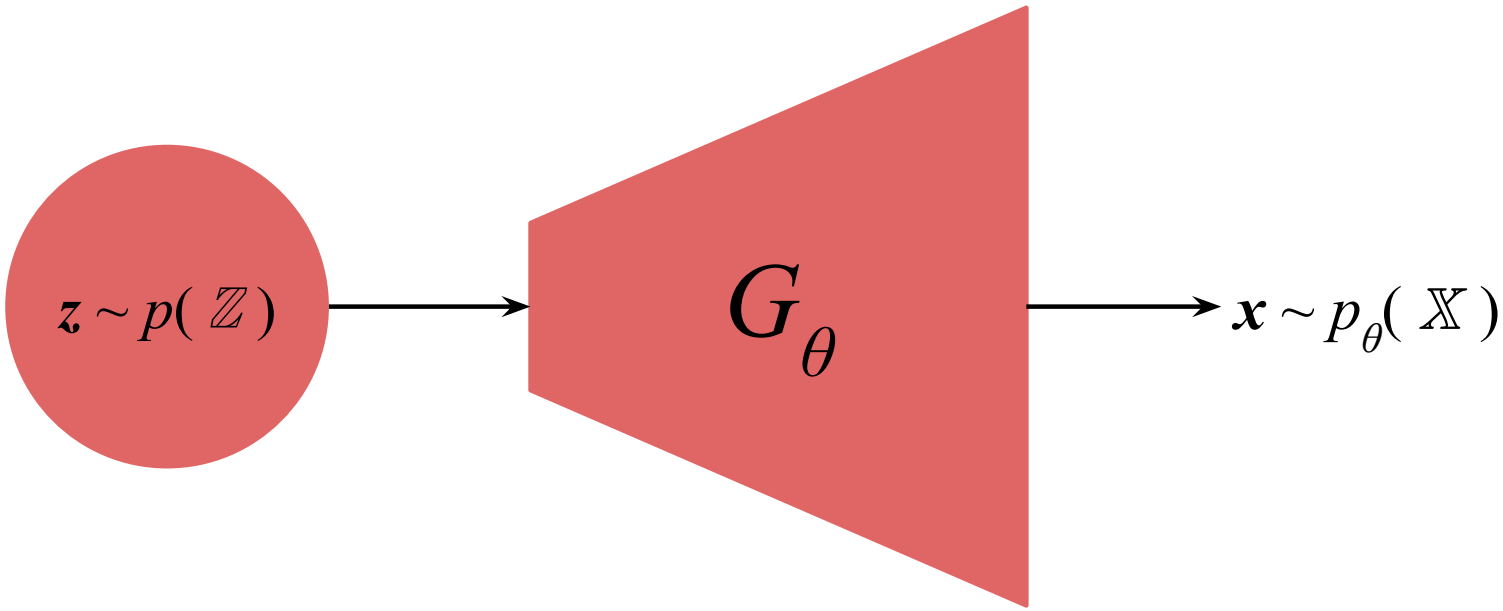
To achieve the above features, we should narrow our selections for $\mathcal{D}$.

Generator

The generator model is a deep neural network characterized by:

- Latent random variable $\mathbb{R}^m \ni \boldsymbol{z} \sim p(\mathbb{Z})$
- Generator network $G_\theta : \mathbb{R}^m \to \mathbb{R}^n$

☞ Because we just need samples of $p_\theta(\mathbb{X})$, there is no limitation over G_θ mapping.



Distance Metric

Labeled Dataset

We have samples from $p_{\text{data}}(\mathbb{X})$ and $p_\theta(\mathbb{X})$ as $\{\boldsymbol{x}_i\}_{i=1}^N$. We label sample $\boldsymbol{x}_i$ as:

$$y_i = \begin{cases} 1 & \text{if} \quad \boldsymbol{x}_i \sim p_{\text{data}}(\mathbb{X}) \\ 0 & \text{if} \quad \boldsymbol{x}_i \sim p_\theta(\mathbb{X}) \end{cases}$$

Thus we have labeled dataset $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$.

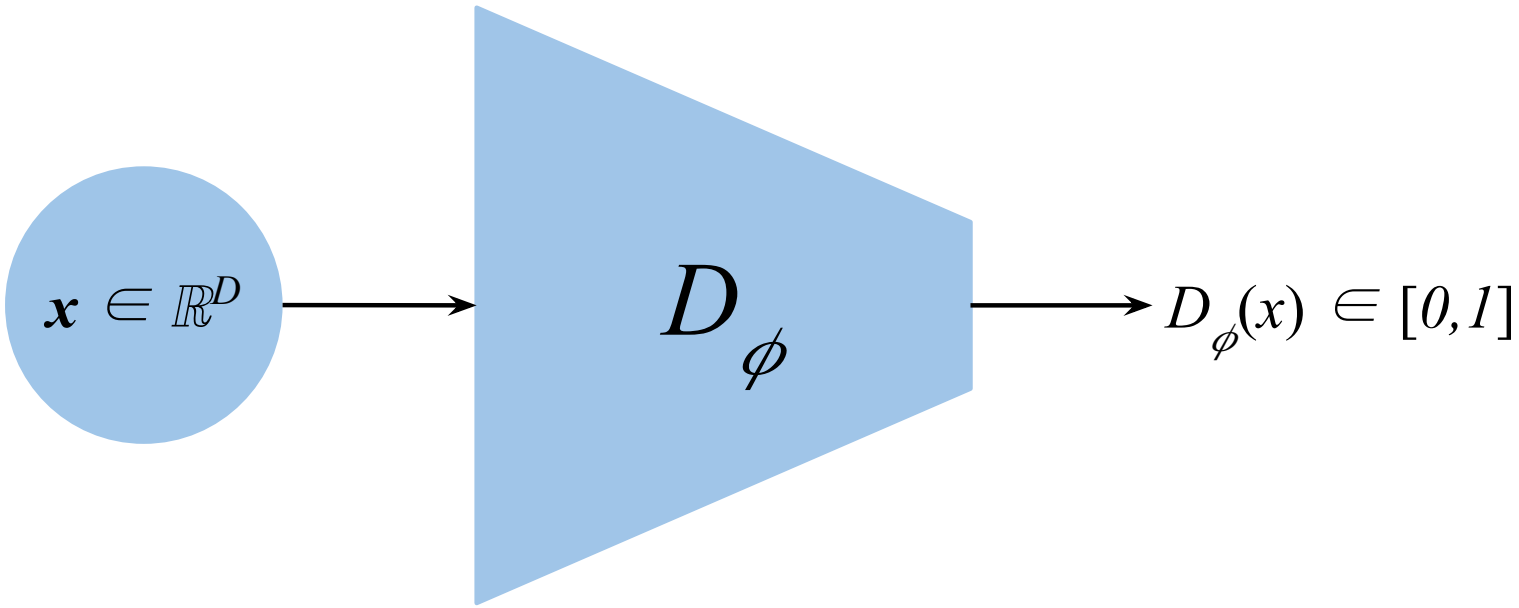
- ☞ Pay attention! The samples with $y = 0$ in the dataset are a function of generator parameters $\boldsymbol{\theta}$.

Distance Metric

Distance Metric Design

To narrow our selection for divergence metic $\mathcal{D}(\cdot, \cdot)$, we assume it to be a classifier that models $p_\phi(Y|\boldsymbol{x}) = \text{Ber}(Y|D_\phi(\boldsymbol{x}))$, known as *Discriminator* characterized by:

- Input vector $\boldsymbol{x} \in \mathbb{R}^D$
- Discriminator network $D_\phi : \mathbb{R}^D \to [0, 1]$



Simplifying the Loss

For Bernoulli distribution, we have:

$$p_\phi(Y|\boldsymbol{x}) = \text{Ber}(Y|D_\phi(\boldsymbol{x})) \Rightarrow p_\phi(y|\boldsymbol{x}) = (D_\phi(\boldsymbol{x}))^y \left(1 - D_\phi(\boldsymbol{x})\right)^{(1-y)}$$

We can write the binary cross entropy loss as:

$$\begin{aligned}\mathbb{E}_{(\boldsymbol{x},y)\sim p(\mathbb{X},Y)} \left[\log p_\phi(y|\boldsymbol{x})\right] =& p(y=1)\mathbb{E}_{p(\mathbb{X}|y=1)} \left[\log p_\phi(y=1|\boldsymbol{x})\right] \\ &+ p(y=0)\mathbb{E}_{p(\mathbb{X}|y=0)} \left[\log p_\phi(y=0|\boldsymbol{x})\right]\end{aligned}$$

which can be simplified as:

$$\mathbb{E}_{(\boldsymbol{x},y)\sim p(\mathbb{X},Y)} \left[\log p_\phi(y|\boldsymbol{x})\right] = \frac{1}{2}\mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\mathbb{X})} \left[\log D_\phi(\boldsymbol{x})\right] + \frac{1}{2}\mathbb{E}_{\boldsymbol{x}\sim p_\theta(\mathbb{X})} \left[\log(1 - D_\phi(\boldsymbol{x}))\right]$$

assuming the class prior densities are equal: $p(y=1) = p(y=0) = 0.5$

Distance Metric

Distance Formulation

We argue the best classifier can be a good distance metric:

$$\mathcal{D}(p_{\text{data}}, p_\theta) = \max_\phi \; \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\mathbb{X})} [\log D_\phi(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\mathbb{X})} [\log(1 - D_\phi(\boldsymbol{x}))]$$

- If $p_\theta = p_{\text{data}}$, then the samples are inseparable, thus $D_\phi(\boldsymbol{x}) \simeq 0.5 \; \forall \boldsymbol{x}$ and distance reaches its minimum as:

$$\mathcal{D}(p_{\text{data}}, p_\theta) = 2 \times \log 0.5$$

- If $p_\theta \neq p_{\text{data}}$, then a ideal classifier will result in $D_\phi(\boldsymbol{x}) = y$, thus:

$$\mathcal{D}(p_{\text{data}}, p_\theta) = 0$$

Distance Metric

Optimum Classifier

Note that in the previous part, we found the optimum classifier corresponding to a fixed generative model parameters $\boldsymbol{\theta}$. Thus:

$$\boldsymbol{\phi}^{\star}(\boldsymbol{\theta}) = \underset{\phi}{\text{argmax}} \; \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\log D_{\phi}(\boldsymbol{x}) \right] + \mathbb{E}_{\boldsymbol{x} \sim p_{\theta}(\mathbb{X})} \left[\log(1 - D_{\phi}(\boldsymbol{x})) \right]$$

General View

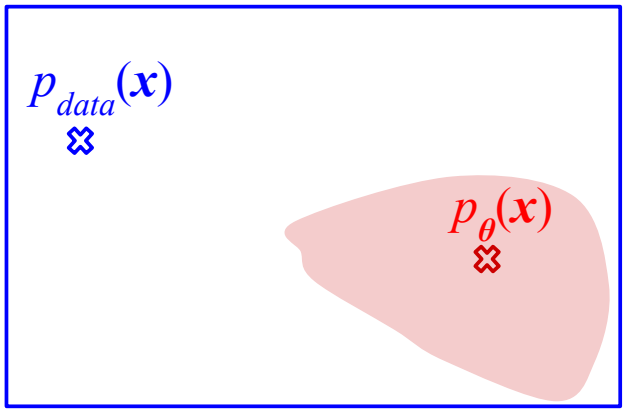


Figure: General view for generative modeling

General View

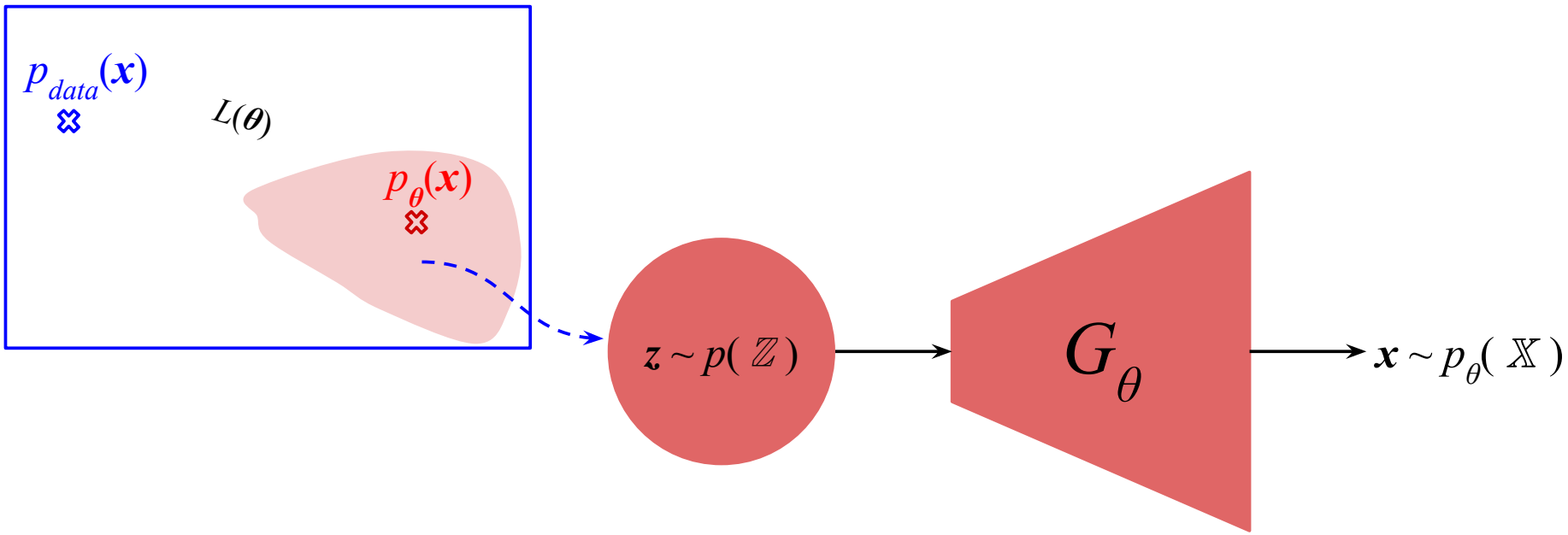


Figure: Charactrizing the parameterized family

General View

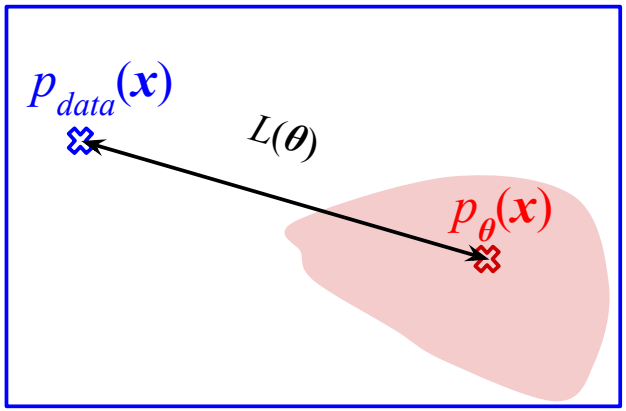


Figure: Charactrizing the distance metric

General View

$$\mathcal{D}(p_{\text{data}}, p_\theta) = \max_\phi \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\mathbb{X})} \left[\log D_\phi(\boldsymbol{x})\right] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\mathbb{X})} \left[\log(1 - D_\phi(\boldsymbol{x}))\right]$$

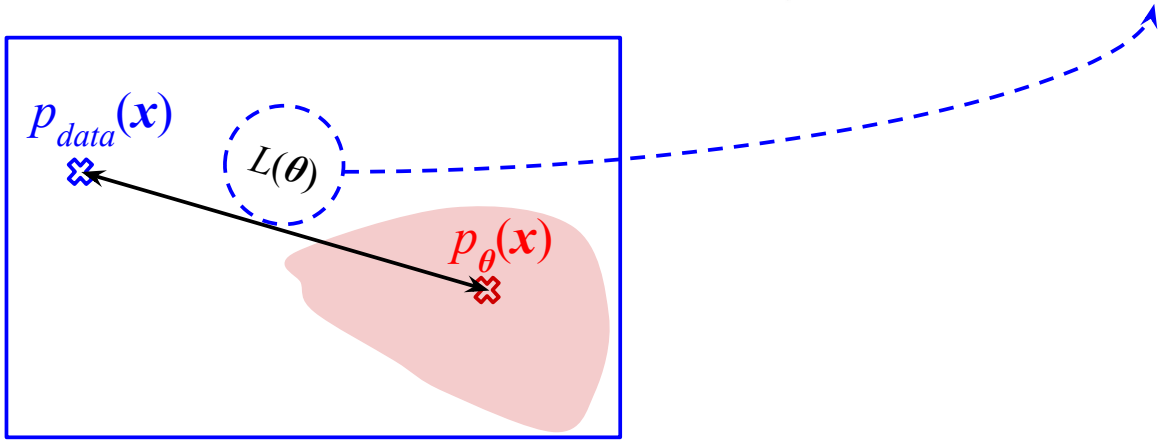


Figure: Charactrizing the distance metric

General View

$$\mathcal{D}(p_{\text{data}}, p_\theta) = \max_\phi \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\mathbb{X})} [\log D_\phi(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\mathbb{X})} [\log(1 - D_\phi(\boldsymbol{x}))]$$

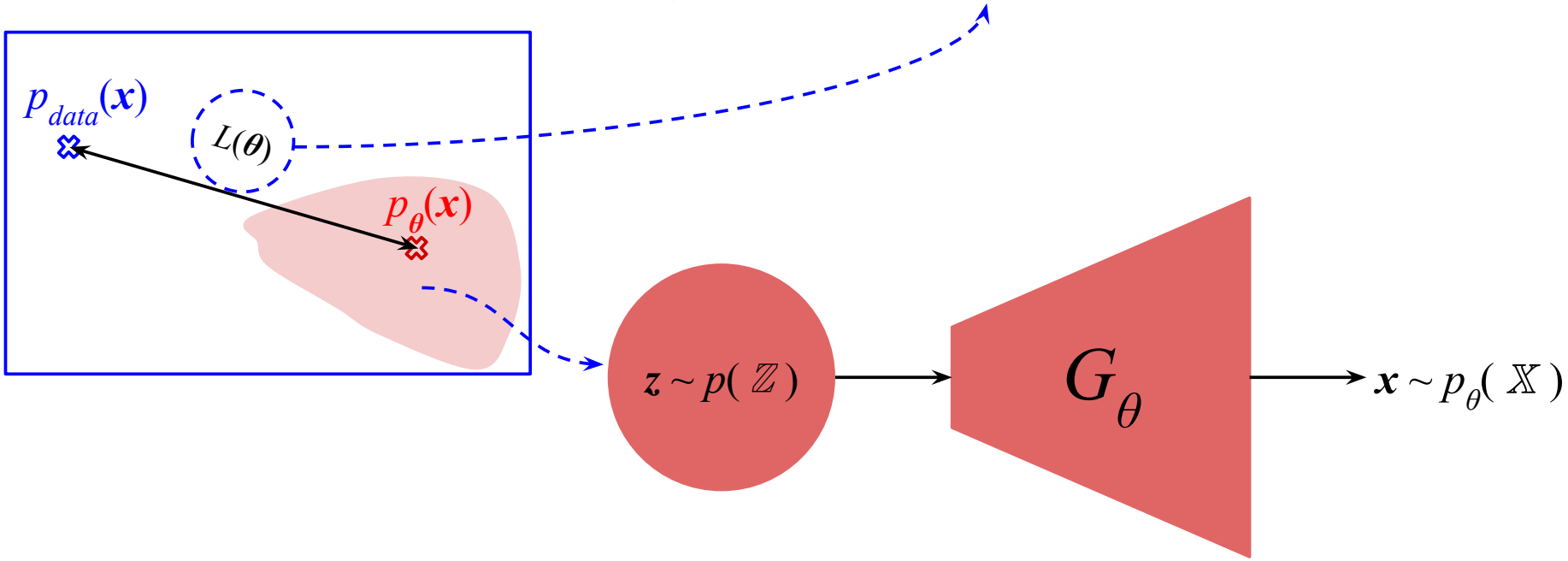


Figure: Generative Adversarial Nets Charactrization

Model Design

Problem

To design the generative model, we should minimize the distance metric, thus the resulting problem is:

$$\min_{\theta} \max_{\phi} V(\boldsymbol{\theta}, \boldsymbol{\phi})$$

where we define:

$$V(\boldsymbol{\theta}, \boldsymbol{\phi}) \triangleq \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\log D_{\phi}(\boldsymbol{x})\right] + \mathbb{E}_{\boldsymbol{x} \sim p_{\theta}(\mathbb{X})} \left[\log(1 - D_{\phi}(\boldsymbol{x}))\right]$$

Generative Adversarial Nets [4]

Justification

$$\min_{\theta} \max_{\phi} \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\log D_{\phi}(\boldsymbol{x})\right] + \mathbb{E}_{\boldsymbol{x} \sim p_{\theta}(\mathbb{X})} \left[\log(1 - D_{\phi}(\boldsymbol{x}))\right]$$

The above approach to create a generative model is called Generative Adversarial Network (GAN) because:

- The target is to train a ***Gnerative*** model (The focus is on Generation task)
- The *Generator* and the *Discriminator* are opposing each other so they are ***Adversarial***.
- Bothe the *Generator* and the *Discriminator* are implemented using neural networks (thus ***Nets***)

Critic Viewpoint [3]

- D_{ϕ} can be seen as a critic who is criticizing the G_{θ} performance.
- By maximization with respect to ϕ, you select the most strict critic.

GAN

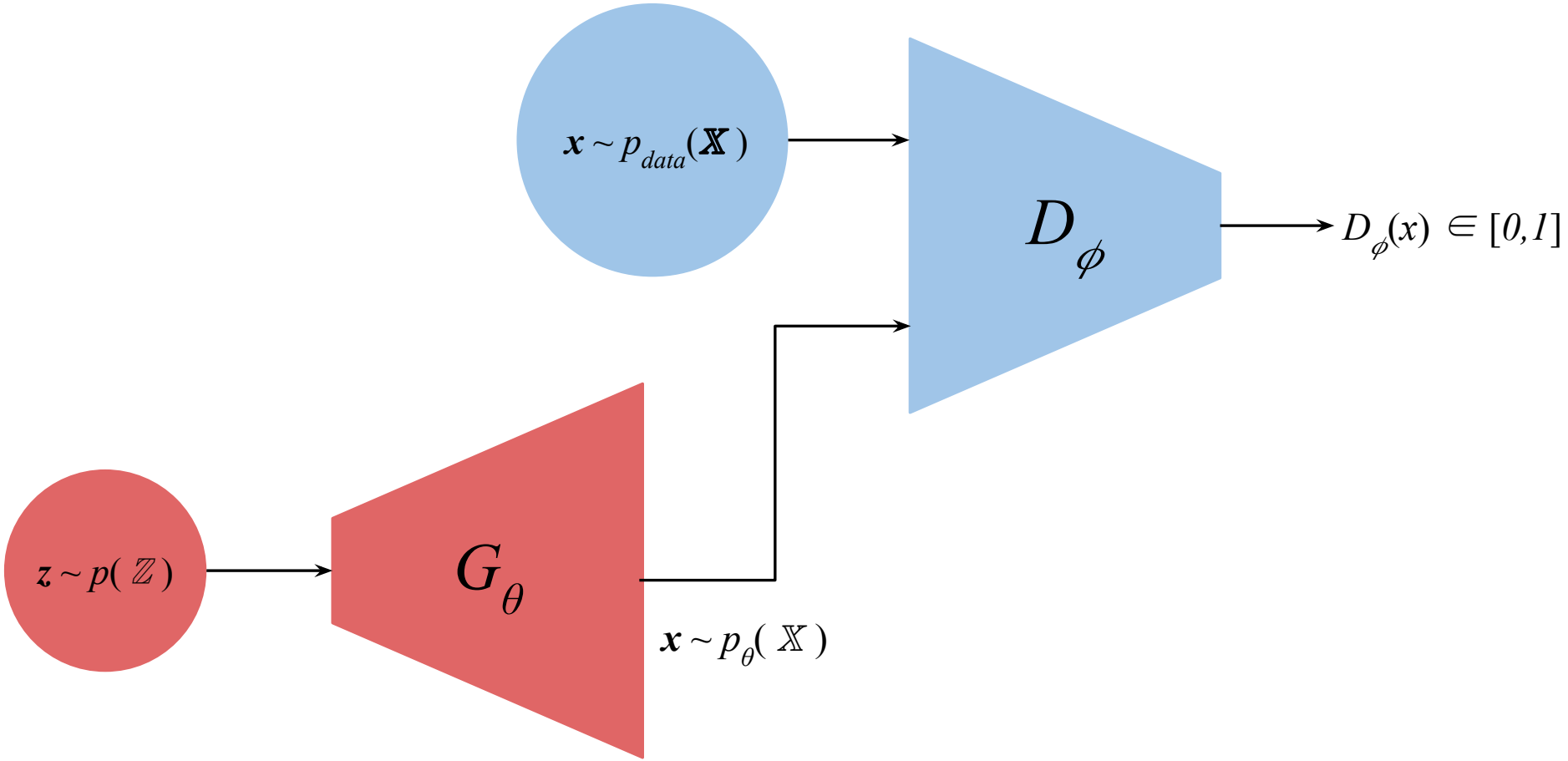


Figure: GAN architecture

Section 3

Global Optimality

Global Optimality

Optimal Discriminator

For a fixed generative model, the optimal discriminator D is:

$$D_\phi^\star(\boldsymbol{x}) = \frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_\theta(\boldsymbol{x})}$$

Thus we have the functional form for the best discriminator (critic) as a function of the generative model.

Limitation

In GAN, we search for the discriminator among the functions implemented by the D_ϕ neural network. Thus we can not necessarily achieve $D_\phi^\star(\boldsymbol{x})$.

Global Optimality

Revisiting Objective

If we plug the optimum discriminator into $V(\boldsymbol{\theta}, \boldsymbol{\phi})$, then we have:

$$
\begin{aligned}
C(\boldsymbol{\theta}) =& \max_{\boldsymbol{\phi}} V(\boldsymbol{\theta}, \boldsymbol{\phi}) \\
=& \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\log D^{\star}_{\phi}(\boldsymbol{x}) \right] + \mathbb{E}_{\boldsymbol{x} \sim p_{\theta}(\mathbb{X})} \left[\log(1 - D^{\star}_{\phi}(\boldsymbol{x})) \right] \\
=& \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\log \frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_{\theta}(\boldsymbol{x})} \right] + \mathbb{E}_{\boldsymbol{x} \sim p_{\theta}(\mathbb{X})} \left[\log \frac{p_{\theta}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_{\theta}(\boldsymbol{x})} \right]
\end{aligned}
$$

where $C(\boldsymbol{\theta})$ is known as *virtual training criterion*, and training becomes:

$$\min_{\boldsymbol{\theta}} C(\boldsymbol{\theta})$$

Limitation

GAN is a likelihood-free model. Equivalently we have no expression to calculate the $p_{\theta}(\boldsymbol{x})$. Thus in practice, you can not calculate or approximate the virtual training criterion.

Virtual Training Criterion

We can simplify the virtual training criterion as:

$$
\begin{aligned}
C(\boldsymbol{\theta}) =& \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\mathbb{X})}\left[\log \frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x})+p_\theta(\boldsymbol{x})}\right] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\mathbb{X})}\left[\log \frac{p_\theta(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x})+p_\theta(\boldsymbol{x})}\right] \\
=& \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\mathbb{X})}\left[\log \frac{p_{\text{data}}(\boldsymbol{x})}{\frac{p_{\text{data}}(\boldsymbol{x})+p_\theta(\boldsymbol{x})}{2}}\right] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\mathbb{X})}\left[\log \frac{p_\theta(\boldsymbol{x})}{\frac{p_{\text{data}}(\boldsymbol{x})+p_\theta(\boldsymbol{x})}{2}}\right] - \log(4) \\
=& \mathrm{KL}\left(p_{\text{data}} \| \frac{p_{\text{data}}+p_\theta}{2}\right) + \mathrm{KL}\left(p_\theta \| \frac{p_{\text{data}}+p_\theta}{2}\right) - \log(4) \\
=& 2 \times \mathrm{JSD}(p_{\text{data}} \| p_\theta) - \log(4)
\end{aligned}
$$

Keeping in mind that the Jensen Shanon Divergence is nonnegative, we conclude:

$$C(\boldsymbol{\theta}) \geq -\log(4)$$

Virtual Training Criterion (Cont.)

In Slide 27, we see:

$$C(\boldsymbol{\theta}) \geq -\log(4)$$

On the other hand assume $\boldsymbol{\theta}^\star$ to be the parameters which result in $p_{\text{data}} = p_\theta$, we have:

$$\begin{aligned} C(\boldsymbol{\theta}^\star) =& \operatorname{KL}\left(p_{\text{data}} \| p_{\text{data}}\right) + \operatorname{KL}\left(p_\theta \| p_\theta\right) - \log(4) \\ =& 0 + 0 - \log(4) \\ =& -\log(4) \end{aligned}$$

Thus $p_{\text{data}} = p_\theta$ is the global minimizer of $C(\boldsymbol{\theta})$.

Section 4

Training Algorithm

Monte Carlo Estimation

We see the objective function for GAN is:

$$\min_{\theta} \max_{\phi} V(\boldsymbol{\theta}, \boldsymbol{\phi})$$

$$V(\boldsymbol{\theta}, \boldsymbol{\phi}) \triangleq \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\log D_{\phi}(\boldsymbol{x})\right] + \mathbb{E}_{\boldsymbol{x} \sim p_{\theta}(\mathbb{X})} \left[\log(1 - D_{\phi}(\boldsymbol{x}))\right]$$

Assume we have access to two sets of samples:

$$\begin{cases} \{\boldsymbol{z}_i\}_{i=1}^{K}, \ \boldsymbol{z}_i \sim p(\mathbb{Z}) \\ \{\boldsymbol{x}_i\}_{i=1}^{K}, \ \boldsymbol{x}_i \sim p_{\text{data}}(\mathbb{X}) \end{cases}$$

Then we can approximate $V(\boldsymbol{\theta}, \boldsymbol{\phi})$ using Monte Carlo estimation:

$$V(\boldsymbol{\theta}, \boldsymbol{\phi}) \simeq \frac{1}{K} \sum_{i=1}^{K} \log D_{\phi}(\boldsymbol{x}_i) + \frac{1}{K} \sum_{i=1}^{K} \log \left(1 - D_{\phi}(G_{\theta}(\boldsymbol{z}_i))\right)$$

Training Algorithm

Algorithm GAN Learning

procedure GAN Learning($\mathcal{D} = \{\boldsymbol{x}_i\}_{i=1}^N, p(\mathbb{Z}), E$)
  Initialization: $\boldsymbol{\theta}, \boldsymbol{\phi}$
  for $e = 1 : E$ **do**
    for $r = 1 : R$ **do**
      Sample minibatch of K latent variable $\{\boldsymbol{z}_1, \dots \boldsymbol{z}_K\}$ from $p(\mathbb{Z})$
      Sample minibatch of K examples $\{\boldsymbol{x}_1, \dots \boldsymbol{x}_K\}$ from Dataset
      $\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} + \mu_\phi \nabla_{\boldsymbol{\phi}} \frac{1}{K} \sum_{i=1}^K \left[\log D_\phi(\boldsymbol{x}_i) + \log\left(1 - D_\phi(G_\theta(\boldsymbol{z}_i))\right)\right]$
    end for
    Sample minibatch of K latent variable $\{\boldsymbol{z}_1, \dots \boldsymbol{z}_K\}$ from $p(\mathbb{Z})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \nabla_{\boldsymbol{\theta}} \frac{1}{K} \sum_{i=1}^K \log\left(1 - D_\phi(G_\theta(\boldsymbol{z}_i))\right)$
  end for
end procedure

Simple Example (Source: [4])

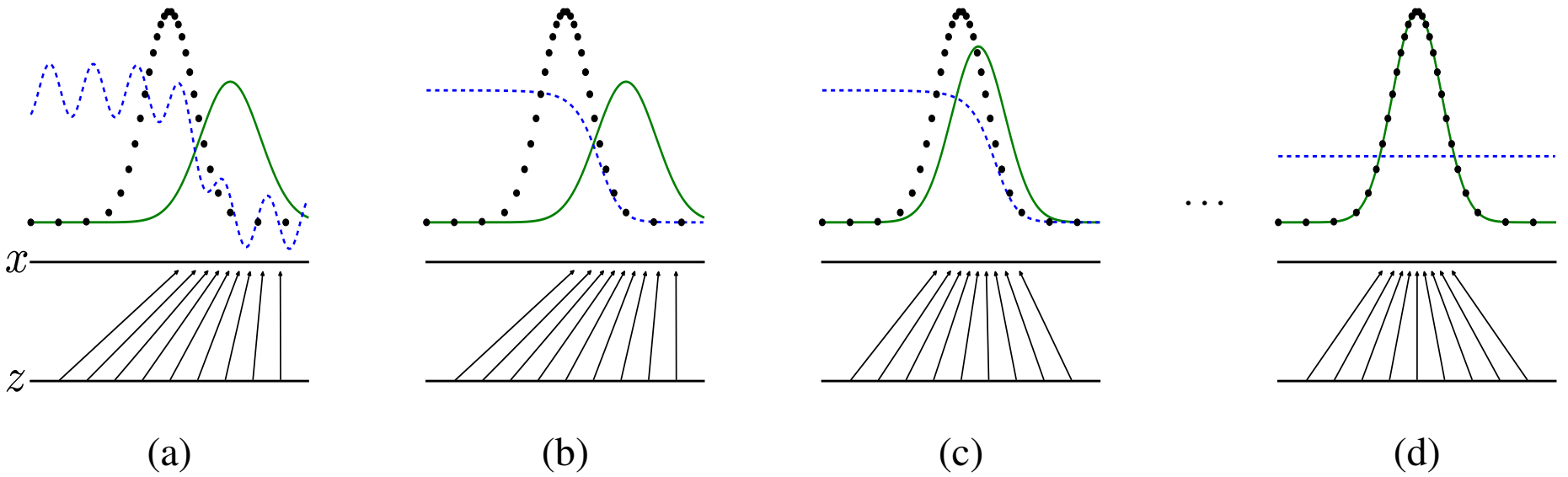
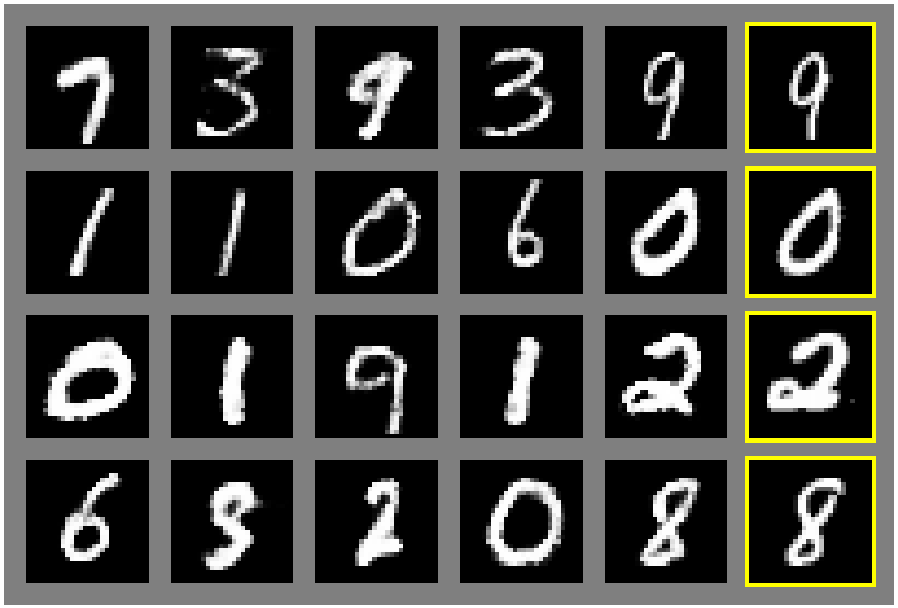
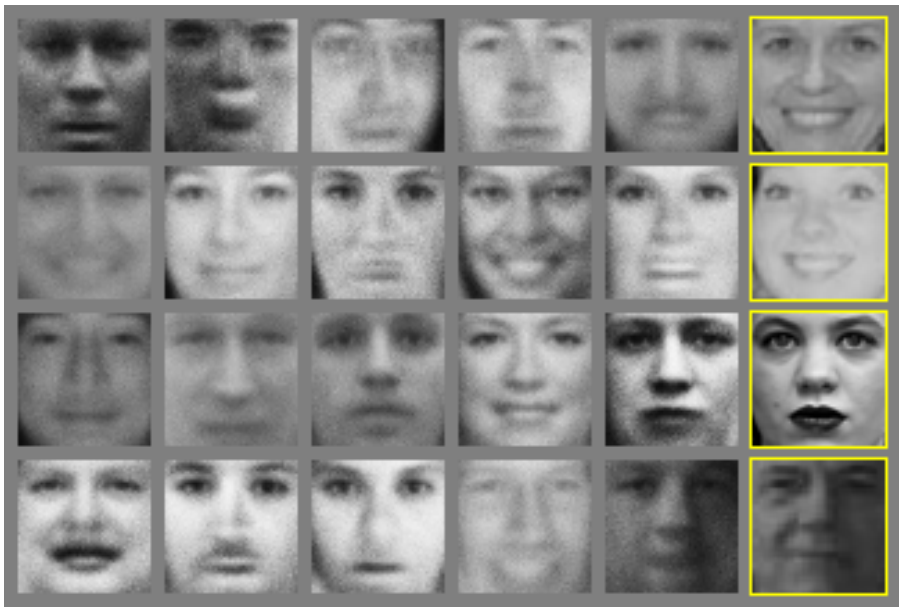


Figure: D (dashed blue line), data samples from p_{data} (dotted black line) and the generative distribution p_θ (solid green line) (a) Near convergence (b) Updating discriminator converging to D $\frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x})+p_\theta(\boldsymbol{x})}$, (c) Updating Generator (gradient of D has guided G_θ to flow to regions that are more likely to be classified as data) (d) After several steps of training.

GAN Samples [4]



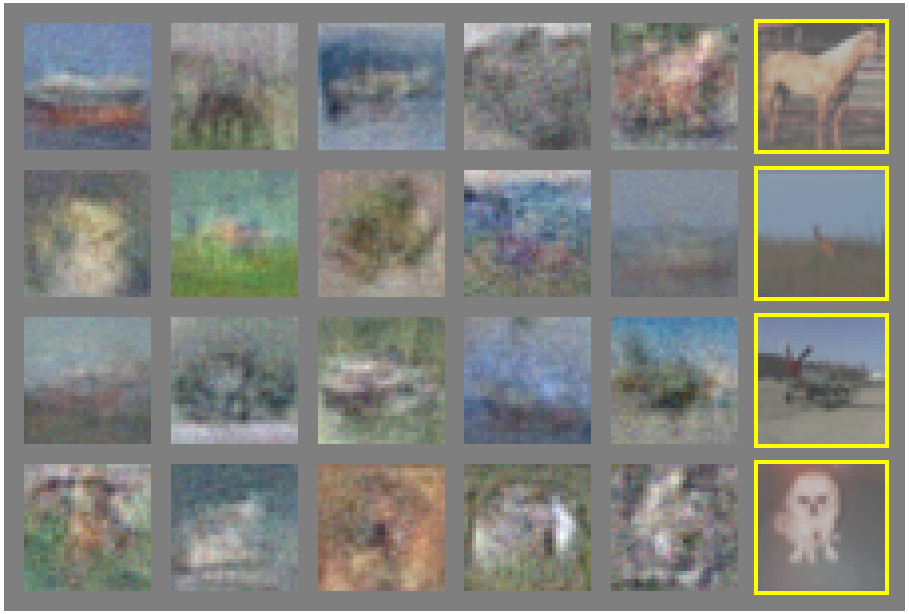
(a) MNIST dataset



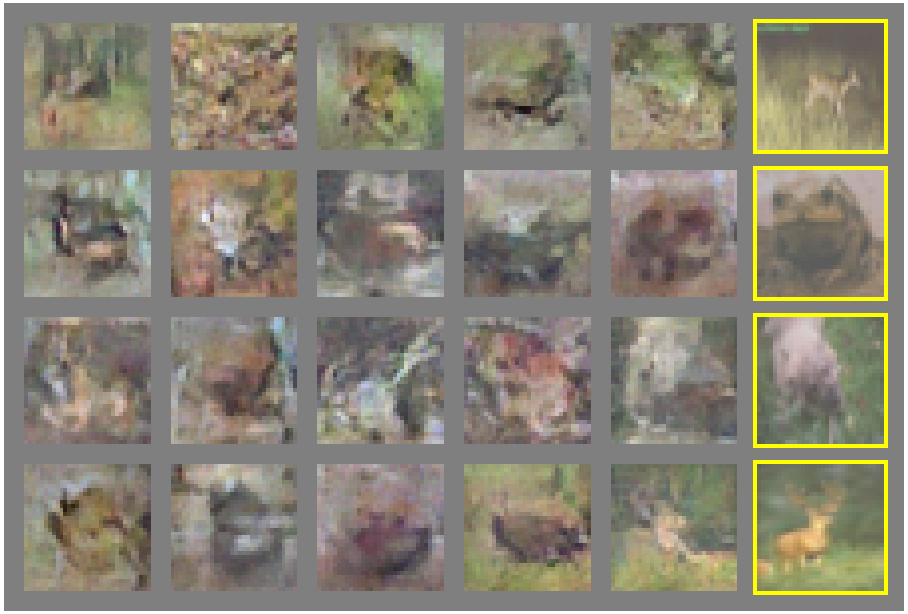
(b) TFD dataset

Figure: Visualization of samples from the model. The rightmost column shows the nearest training example of the neighboring sample, to demonstrate that the model has not memorized the training set.

GAN Samples [4]



(a) CIFA10 dataset (fully-connected)



(b) CIFA10 dataset (convolutional)

Figure: Visualization of samples from the model. The rightmost column shows the nearest training example of the neighboring sample, to demonstrate that the model has not memorized the training set.

Section 5

Deep Convolutional GAN

Generator Architecture (Source: [5])

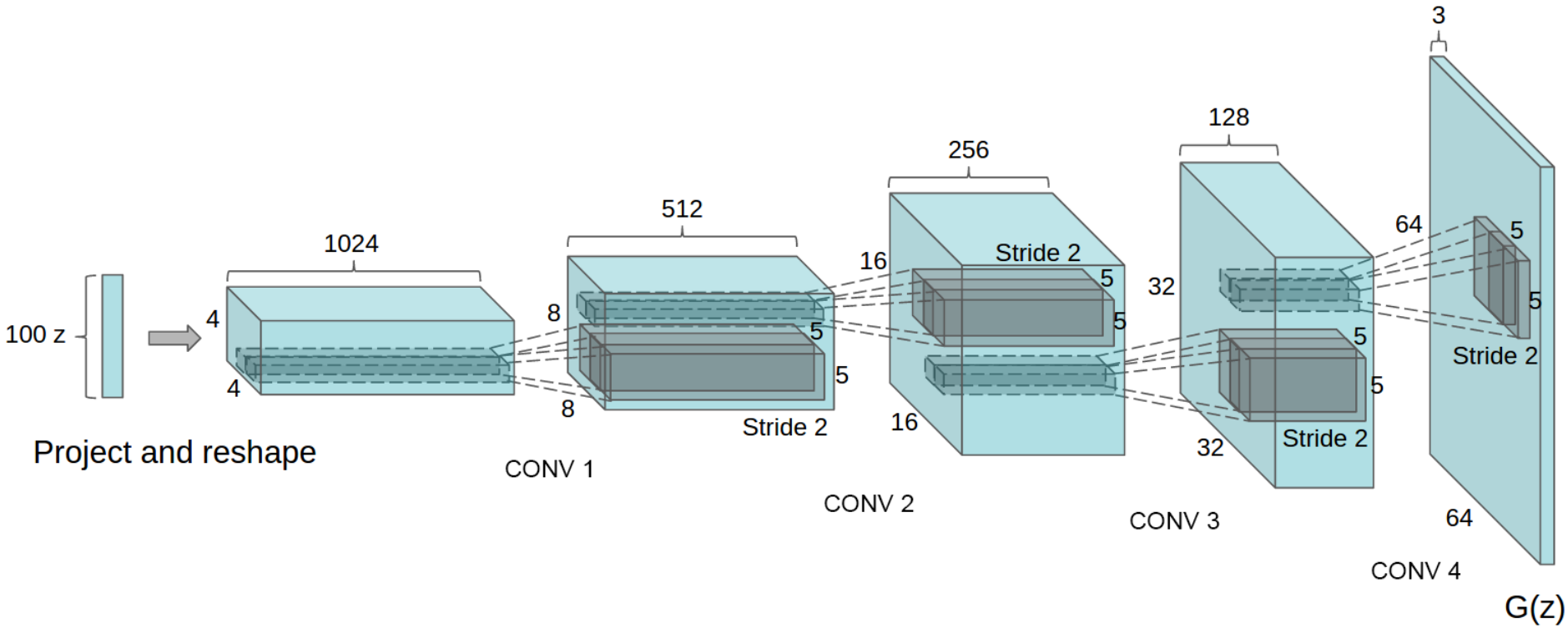


Figure: Generator Architecture in DCGAN (input: $\boldsymbol{z} \in \mathbb{R}^{100}$, output: $G_\theta(\boldsymbol{z}) \in \mathbb{R}^{64 \times 64 \times 3}$)

Discriminator Architecture (Source: [5])

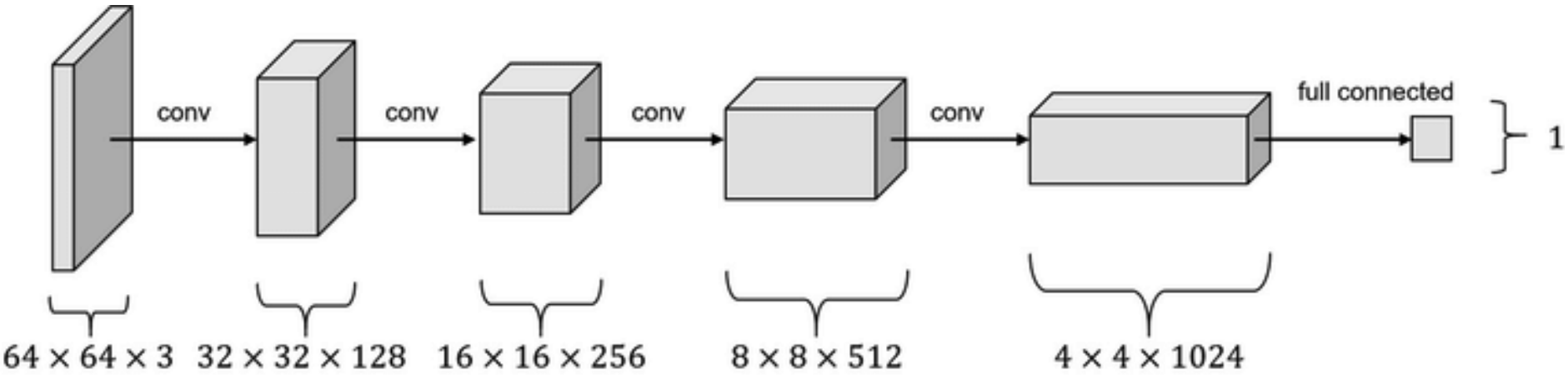


Figure: Discriminator Architecture in DCGAN (input: $\boldsymbol{x} \in \mathbb{R}^{64\times64\times3}$, output: $D_\phi(\boldsymbol{x}) \in [0, 1]$) [6]

Generated Samples (Source: [5])

Figure: Generated bedrooms after one training pass through the dataset.

Generated Samples (Source: [5])

Figure: Generated bedrooms after five training passes through the dataset.

Attribute Vectors in GAN (Source: [5])

SW: Smiling Woman
NW: Neutral Woman
SM: Smiling Man
NM: Neutral Man

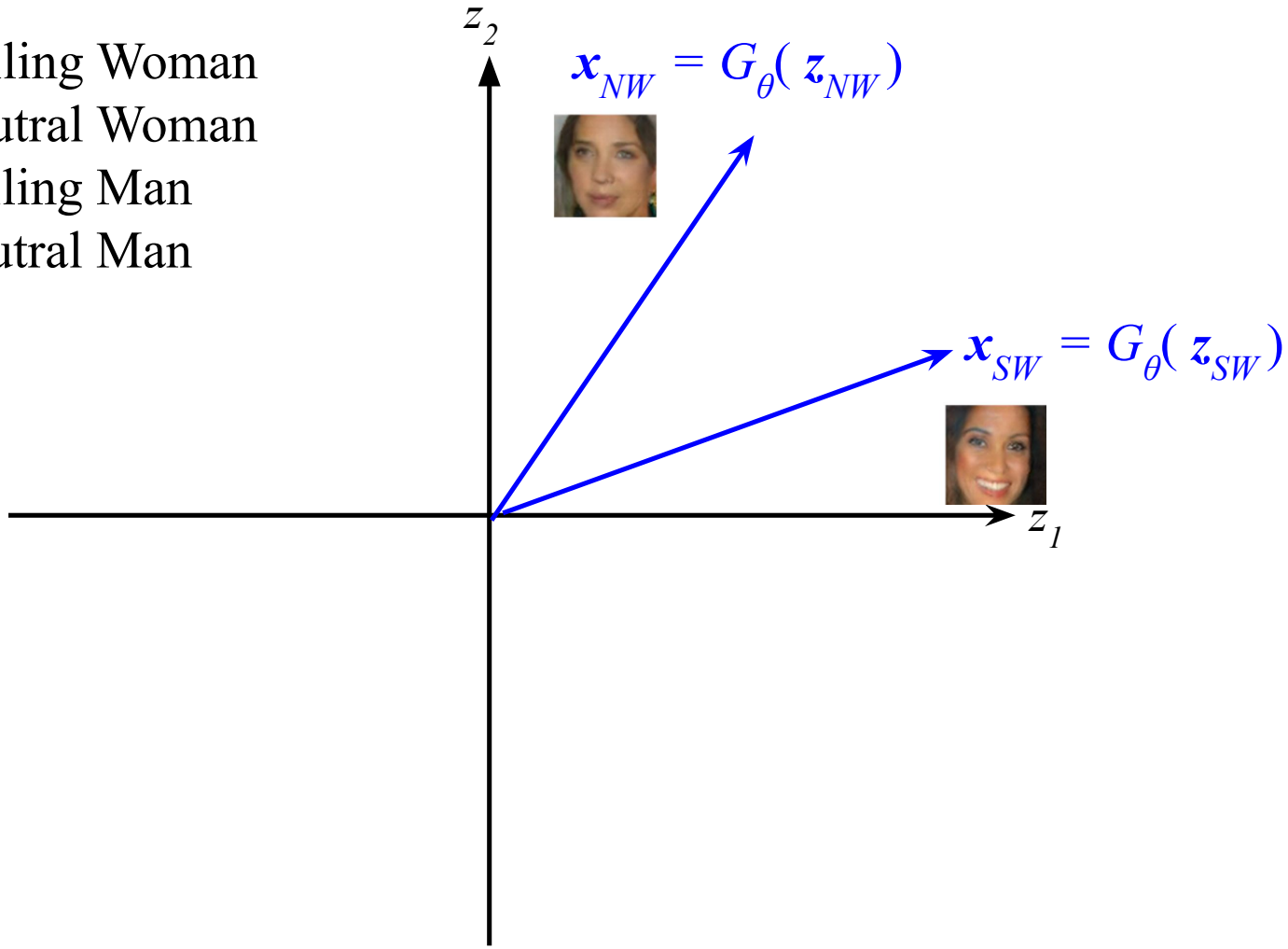


Figure: Smile attribute detection in the latent space

Attribute Vectors in GAN (Source: [5])

SW: Smiling Woman
NW: Neutral Woman
SM: Smiling Man
NM: Neutral Man

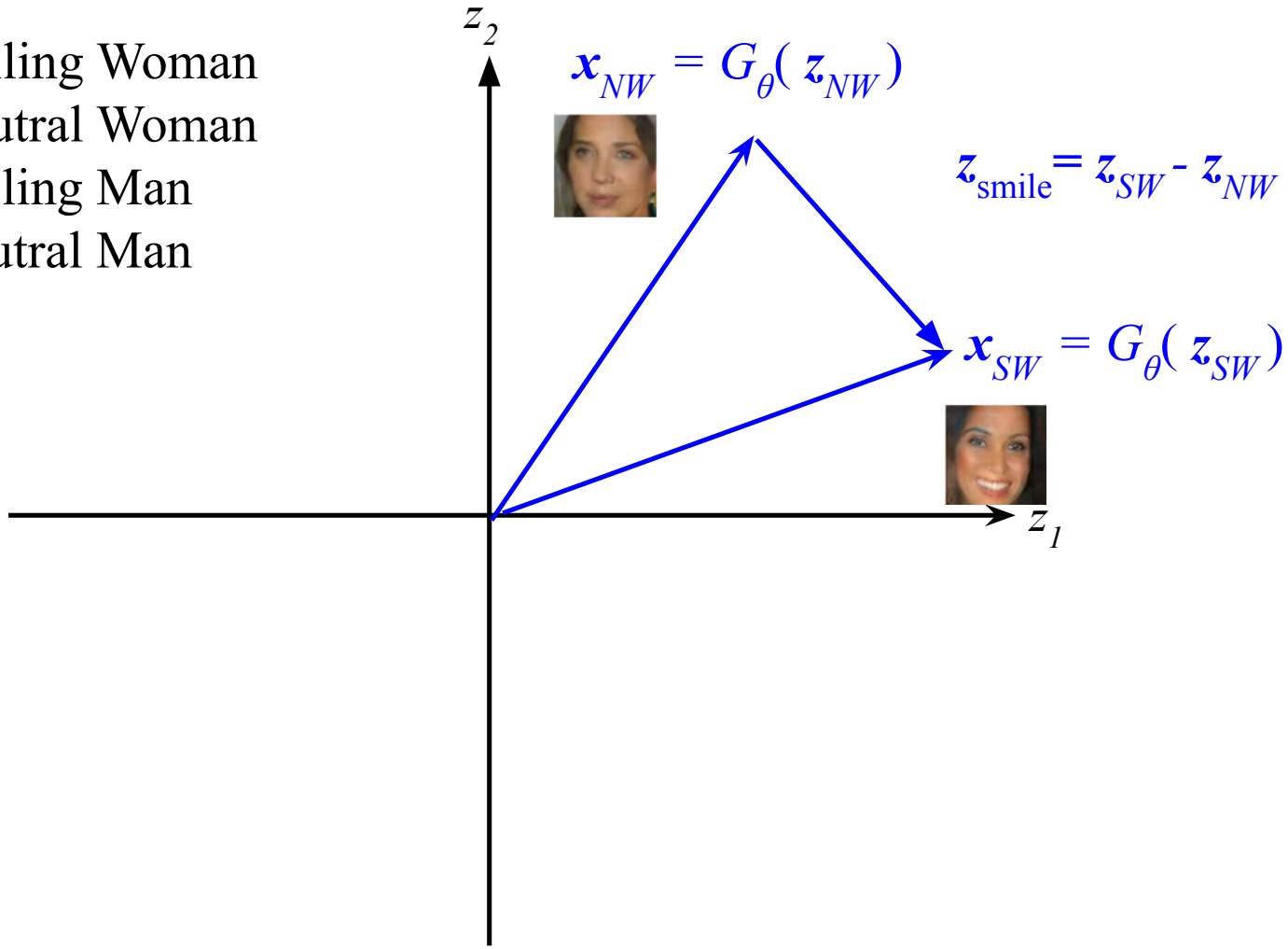


Figure: Smile attribute detection in the latent space

Attribute Vectors in GAN (Source: [5])

SW: Smiling Woman
NW: Neutral Woman
SM: Smiling Man
NM: Neutral Man

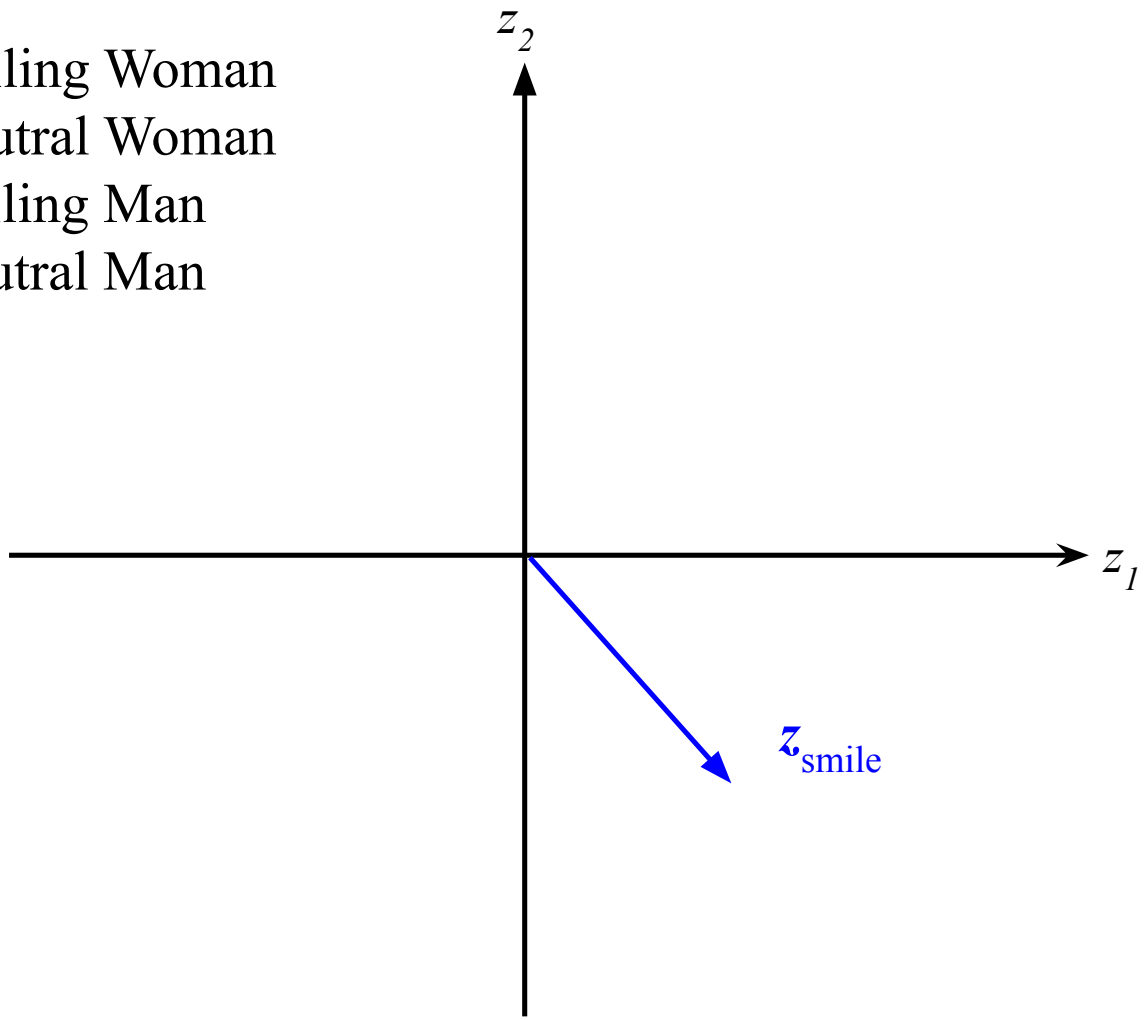


Figure: Smile attribute detection in the latent space

Attribute Vectors in GAN (Source: [5])

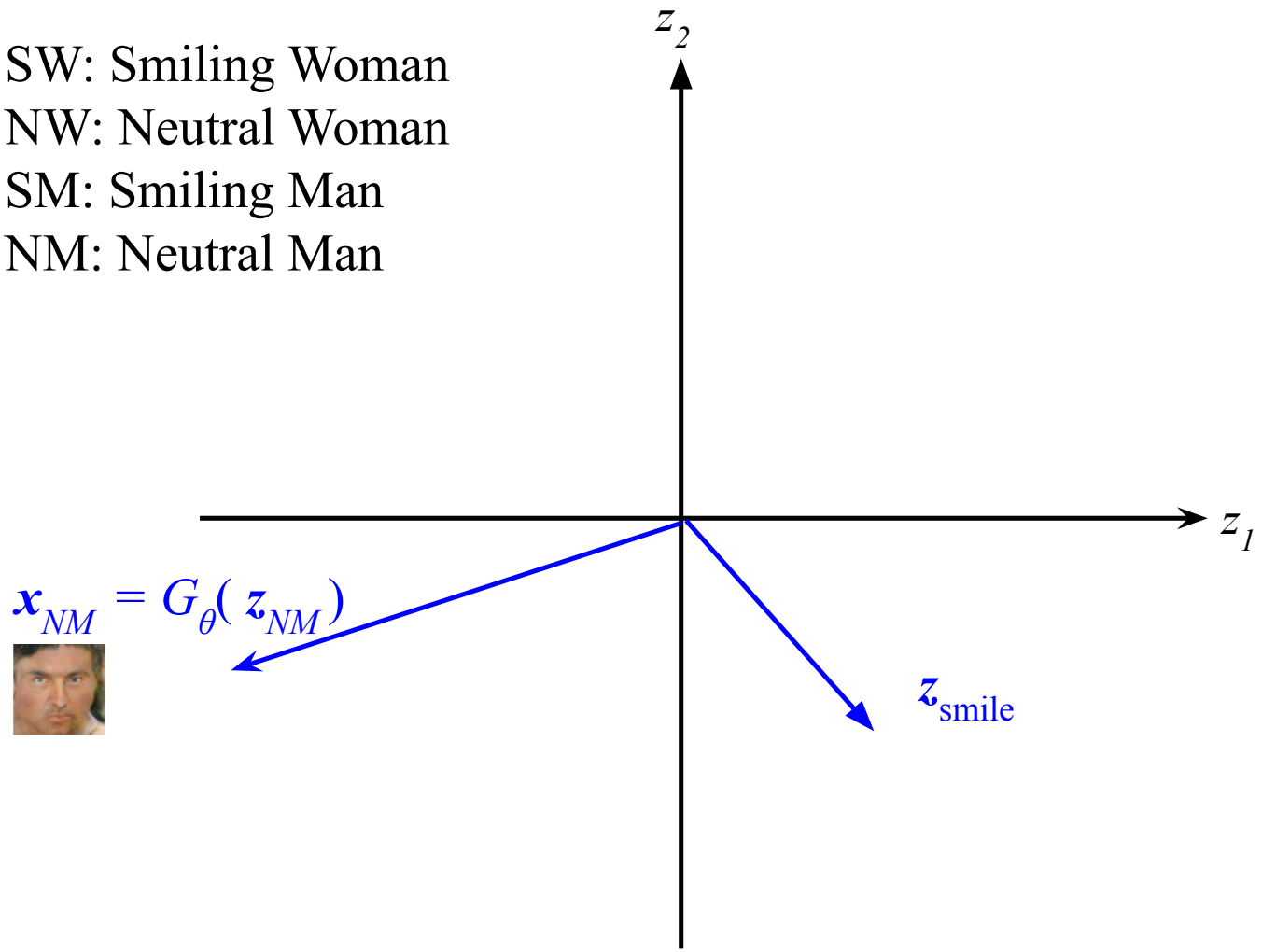


Figure: Smile attribute detection in the latent space

Attribute Vectors in GAN (Source: [5])

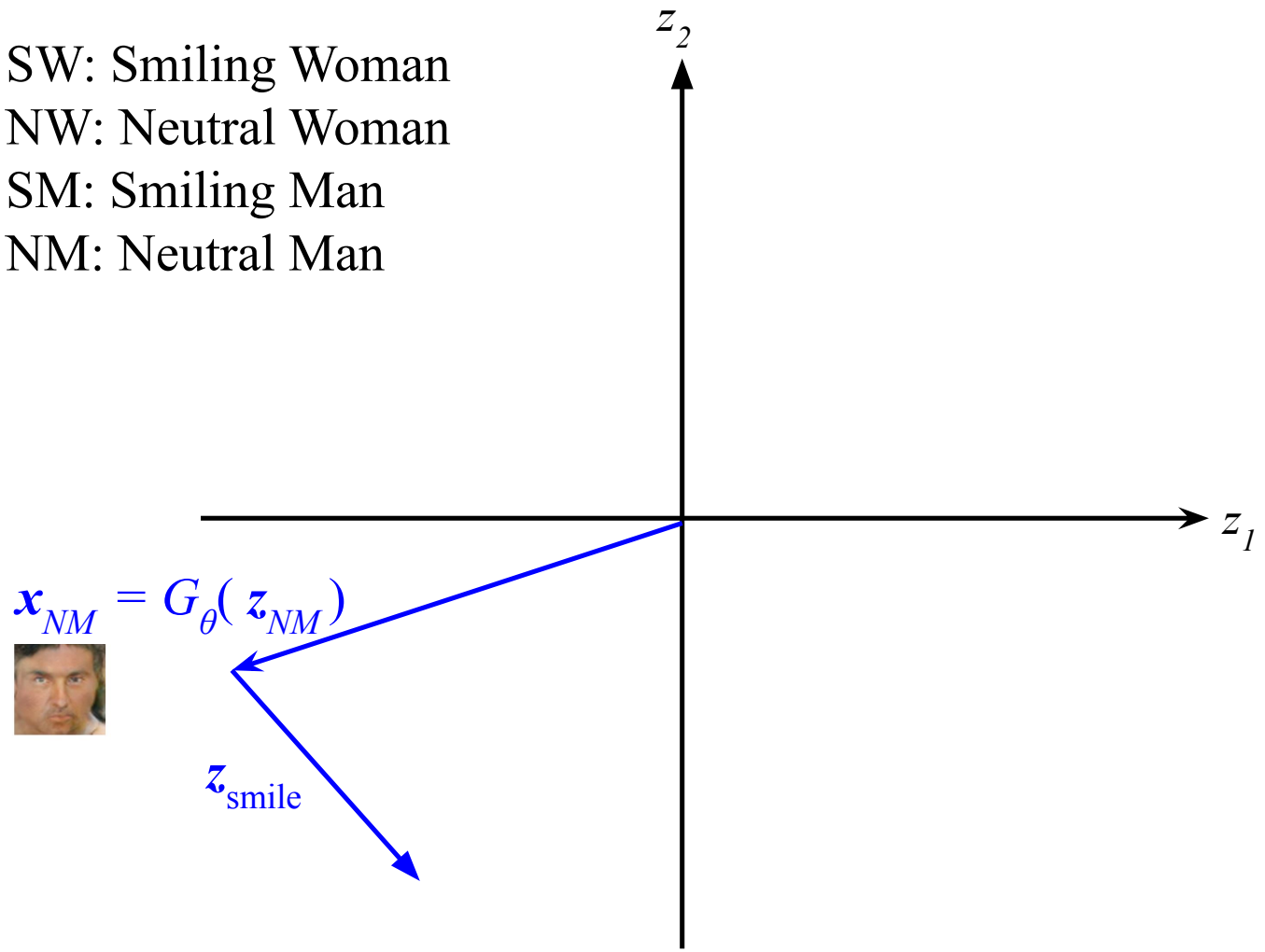


Figure: Smile attribute detection in the latent space

Attribute Vectors in GAN (Source: [5])

SW: Smiling Woman
NW: Neutral Woman
SM: Smiling Man
NM: Neutral Man

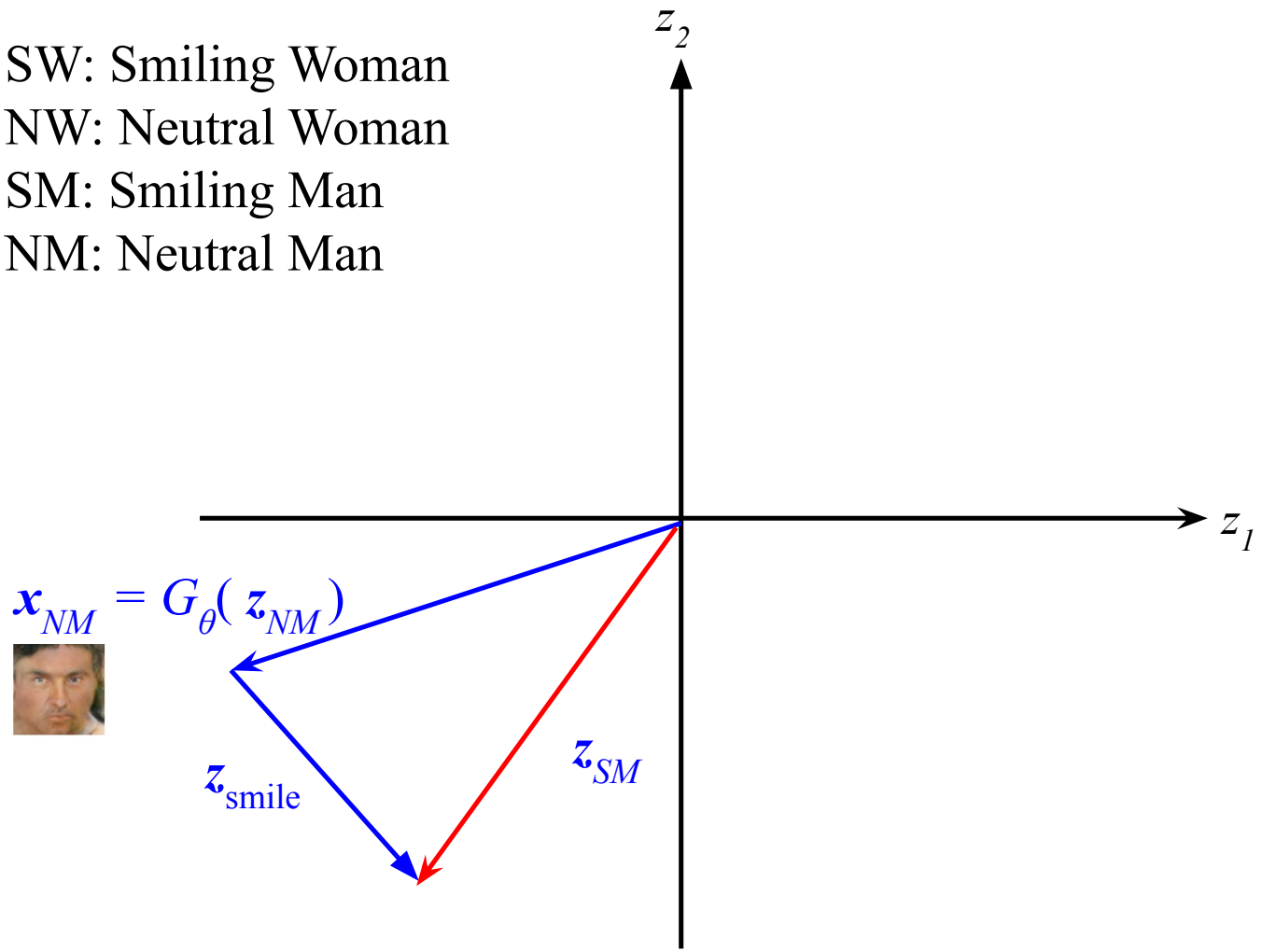


Figure: Smile attribute detection in the latent space

Attribute Vectors in GAN (Source: [5])

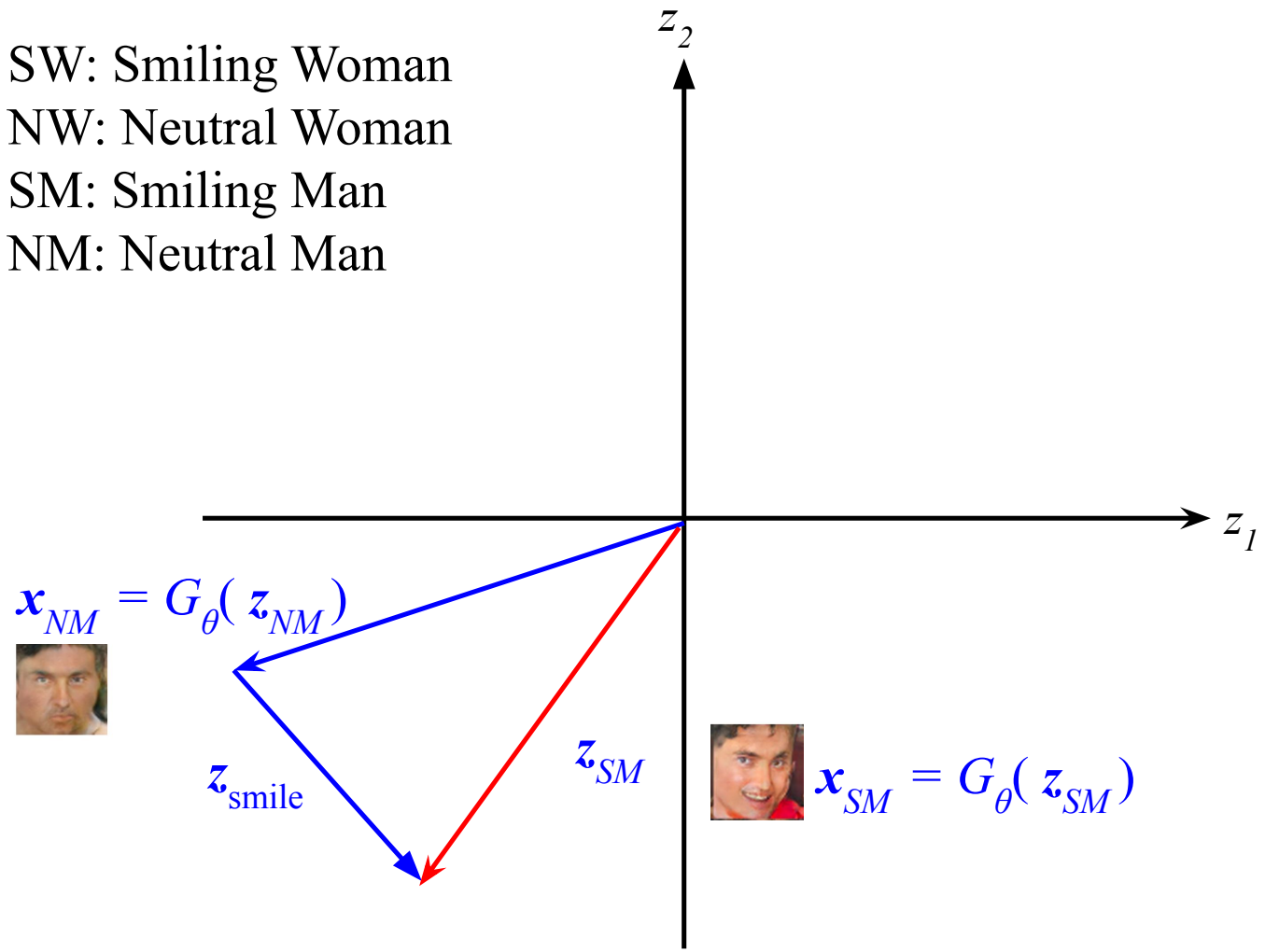


Figure: Smile attribute detection in the latent space

Attribute Vectors in GAN (Source: [5])

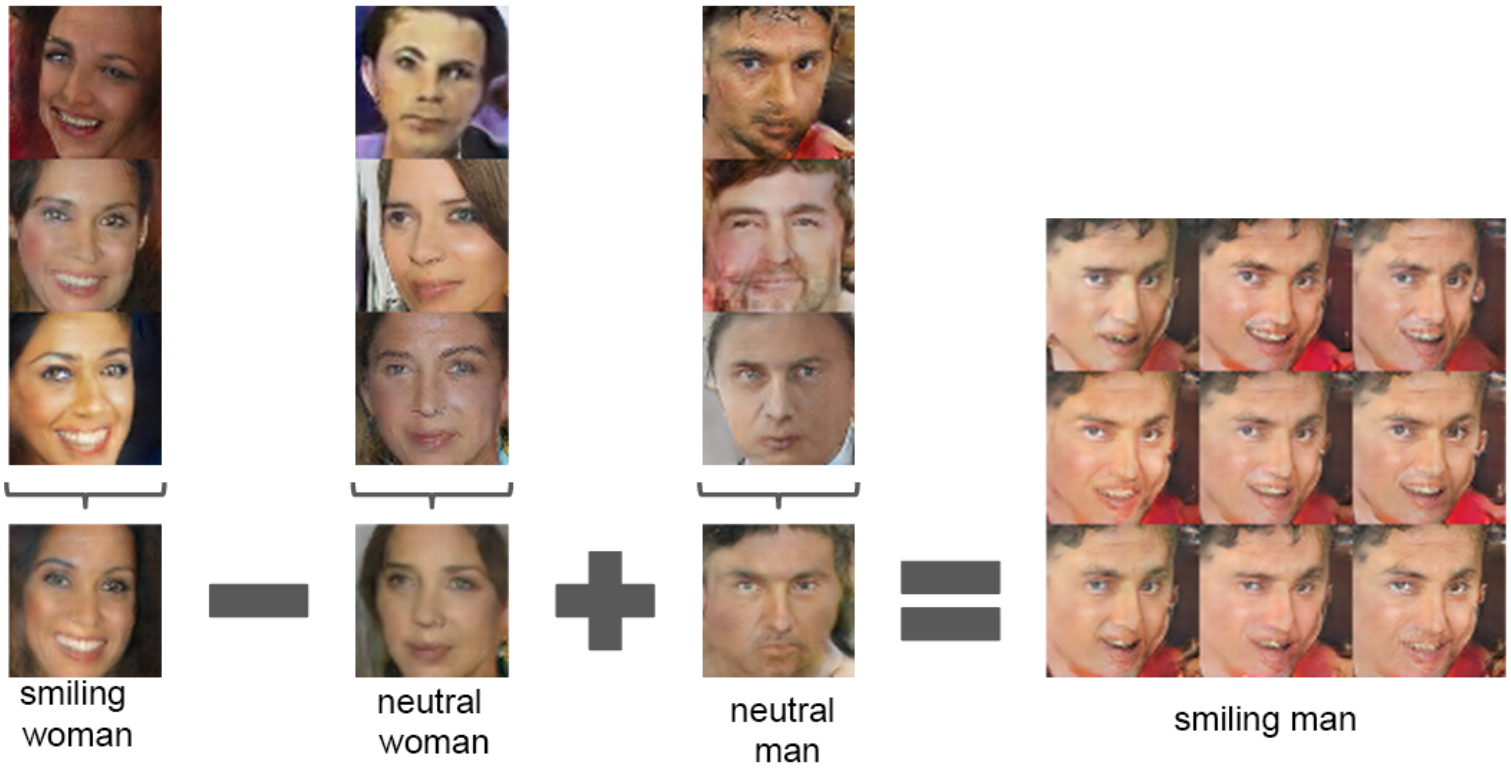


Figure: Converting *Neutral* face into *Smiling* one using latent manipulation

Attribute Vectors in GAN (Source: [5])

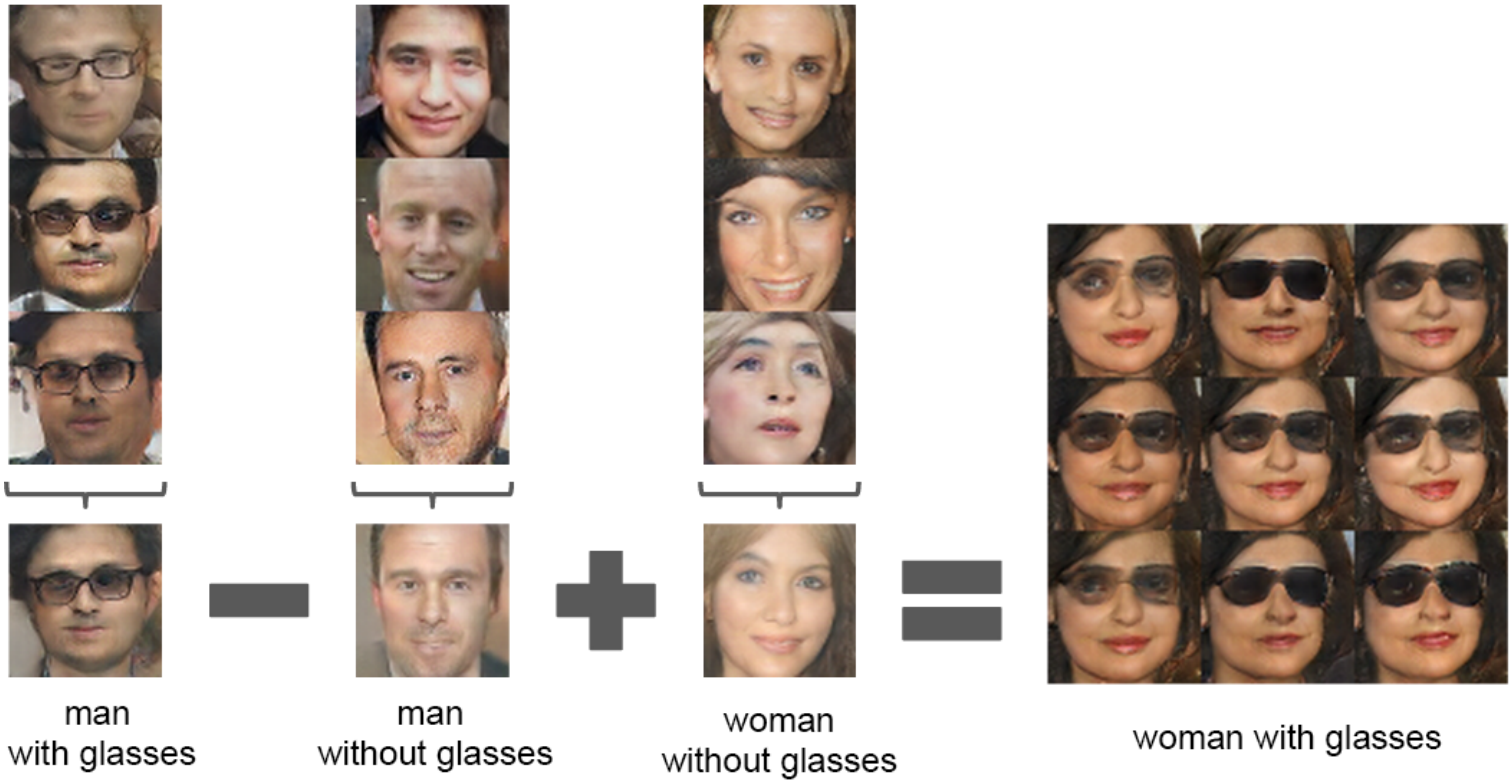


Figure: Converting *without Glasses* face into *with Glasses* one using latent manipulation

Section 6

Mode Collapse

Mode Collapse in GAN

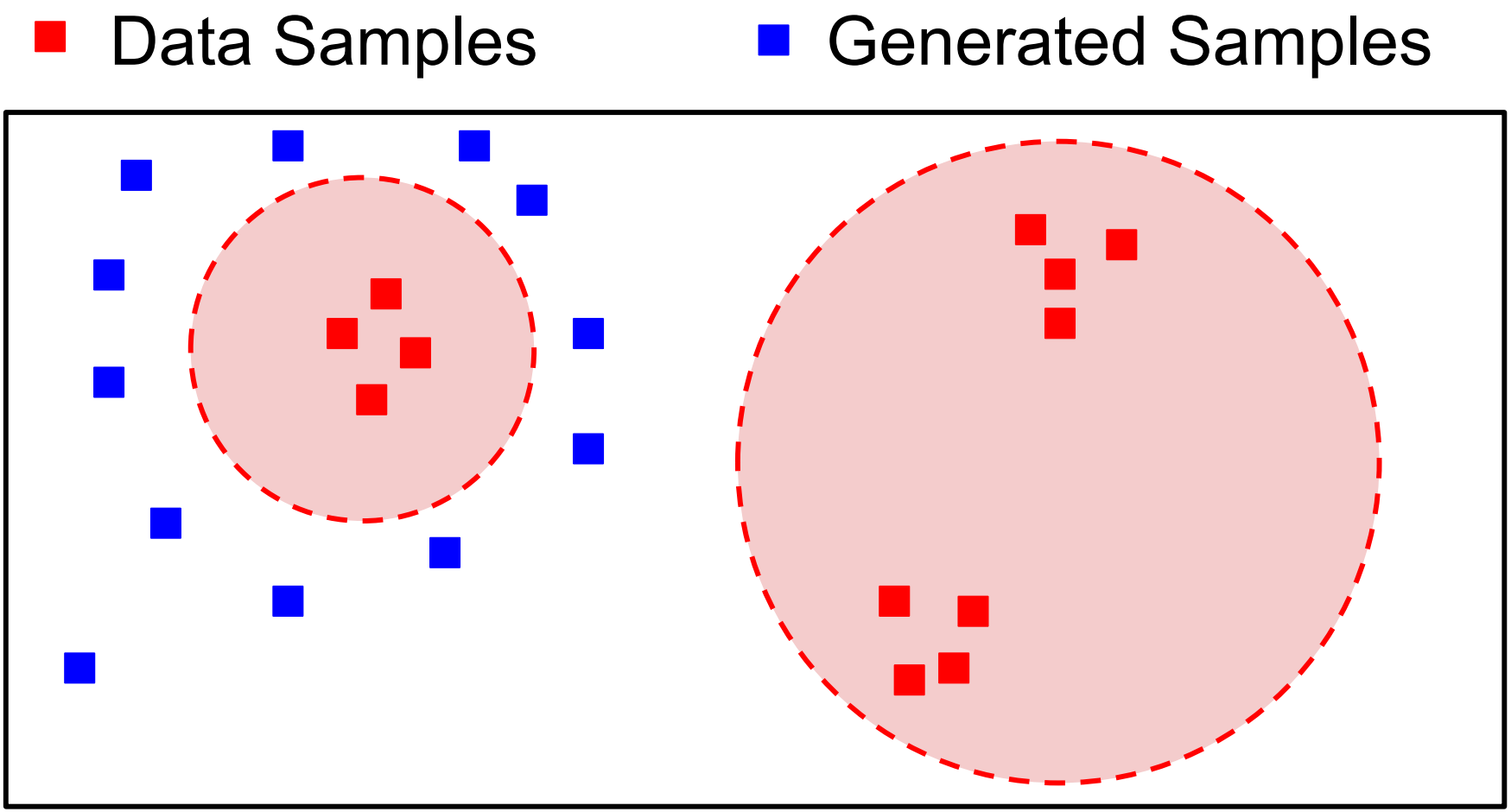


Figure: Classification regions after updating Discriminator (Red region corresponds to 1 in discriminator while the other region show 0 class of discriminator)

Mode Collapse in GAN

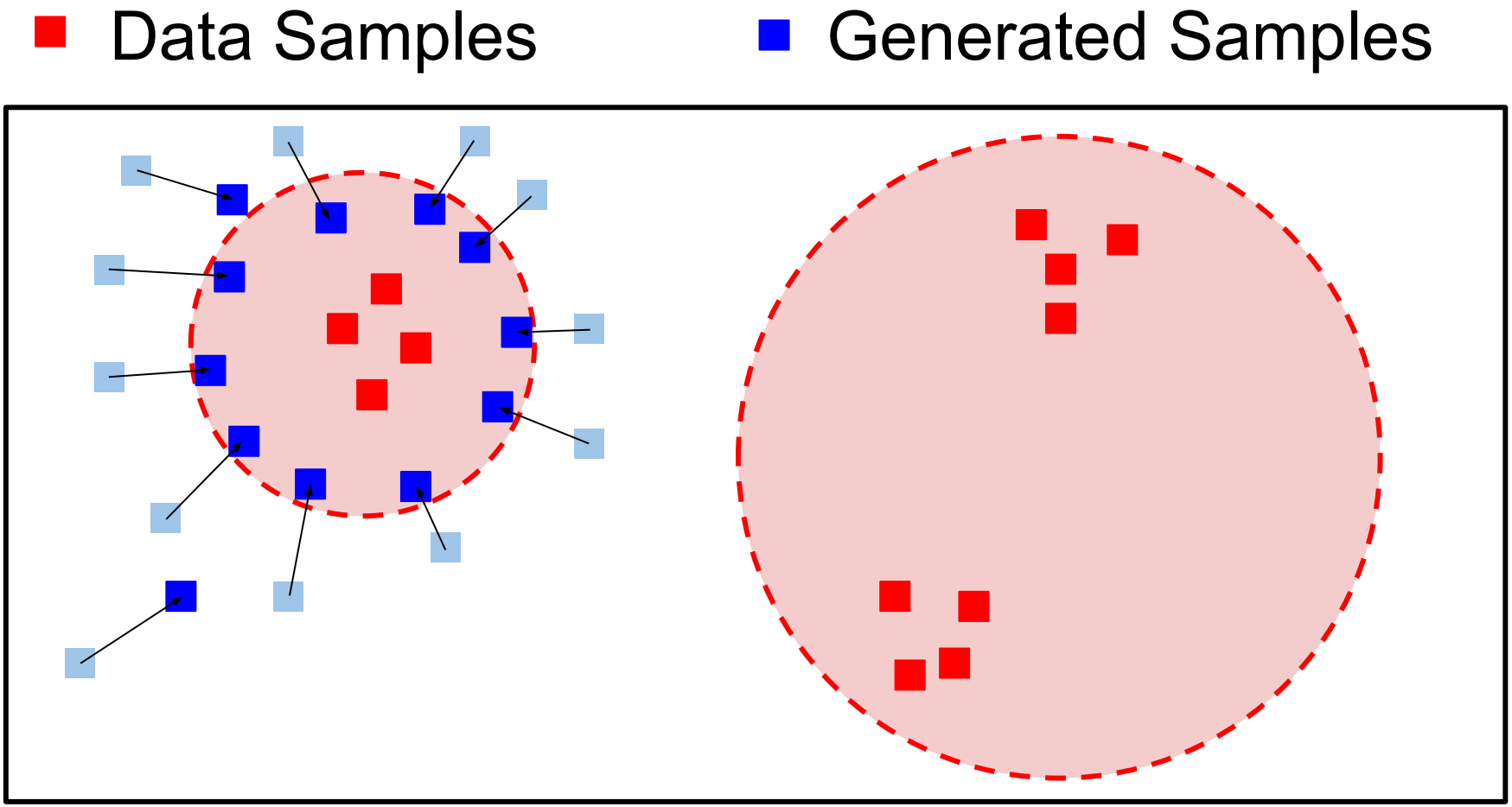


Figure: Generated samples after updating Generator

Mode Collapse in GAN

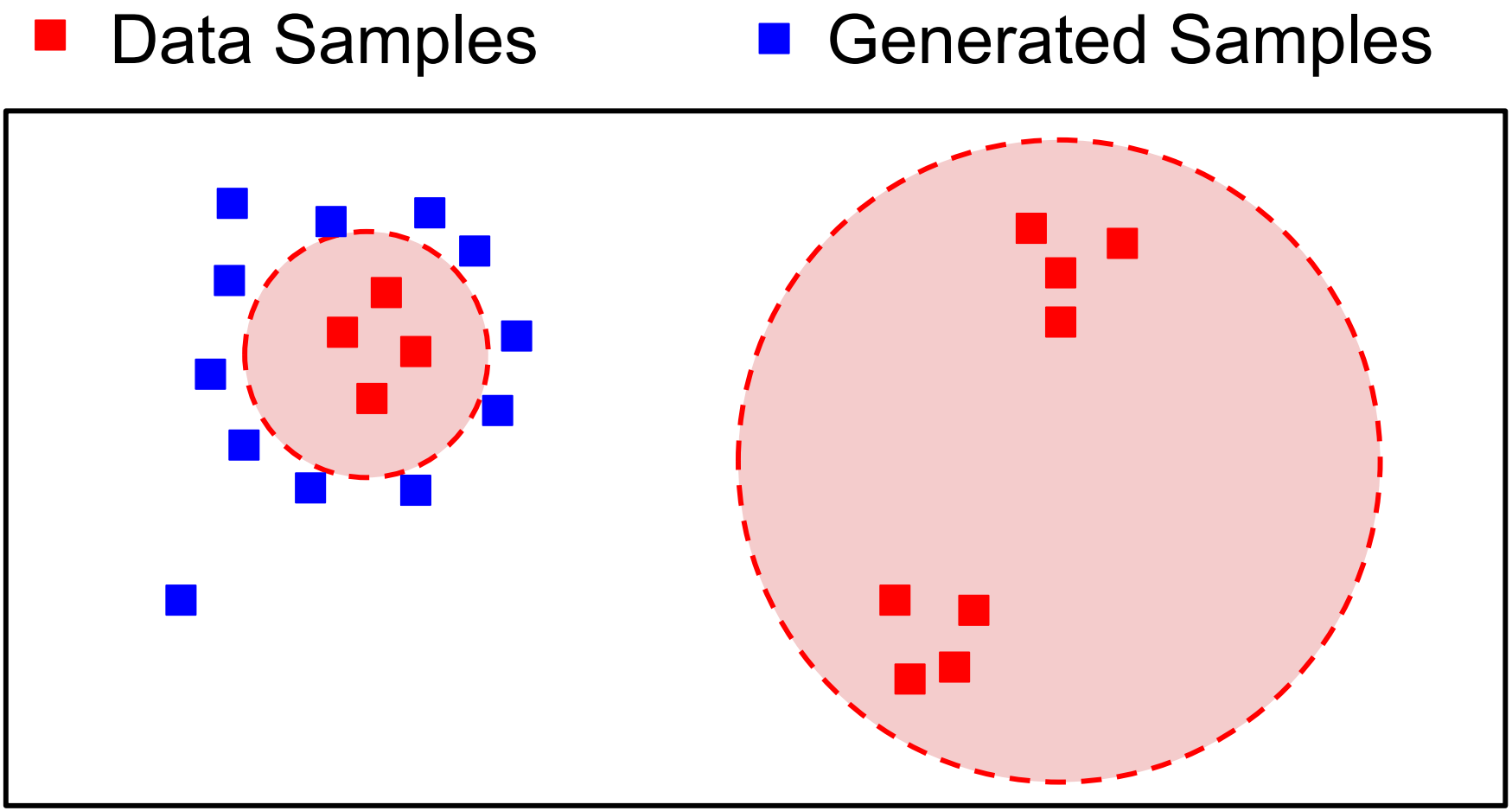


Figure: Classification regions after updating Discriminator

Mode Collapse in GAN

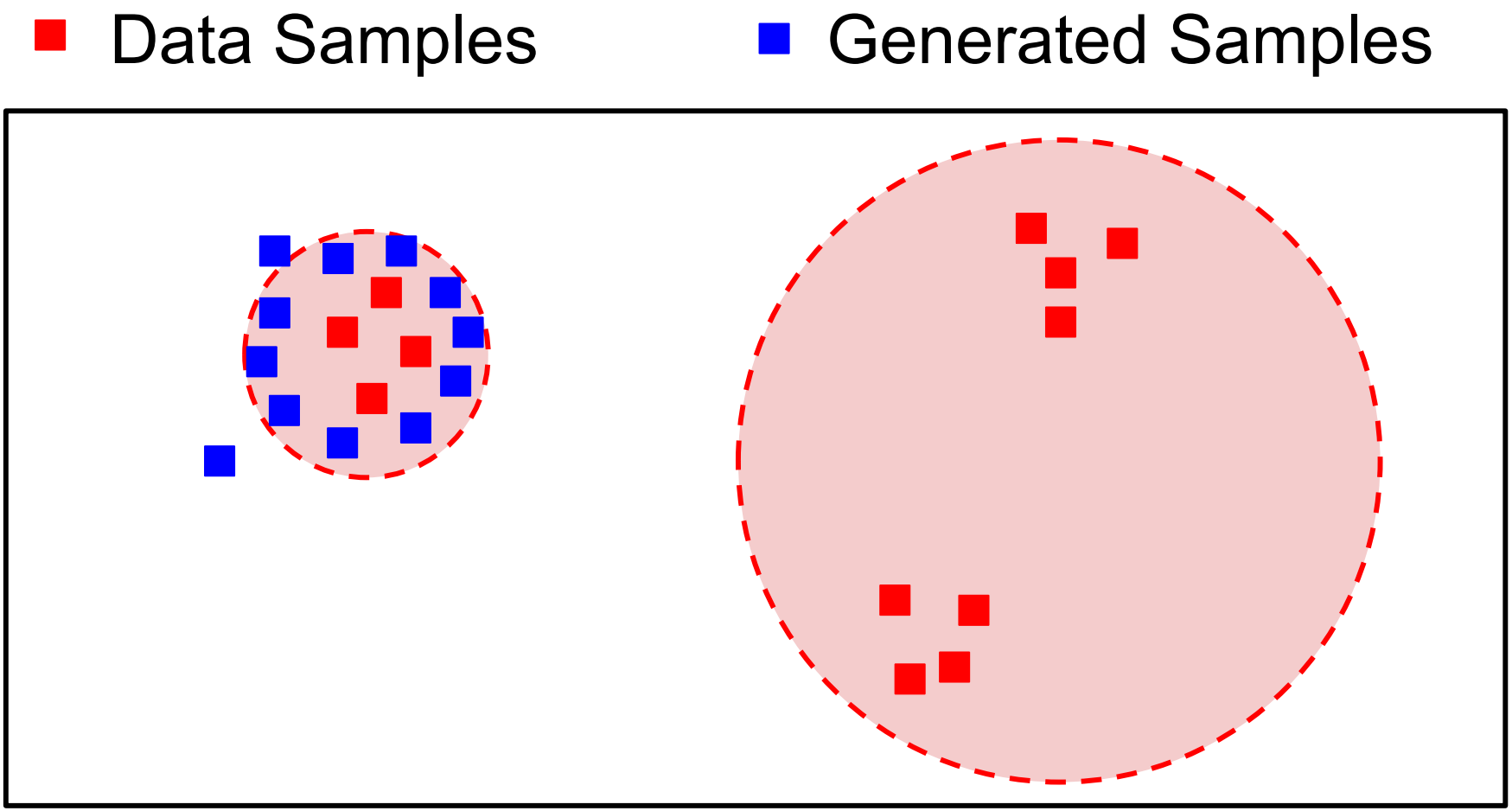


Figure: Generated samples after updating Generator

Mode Collapse in GAN

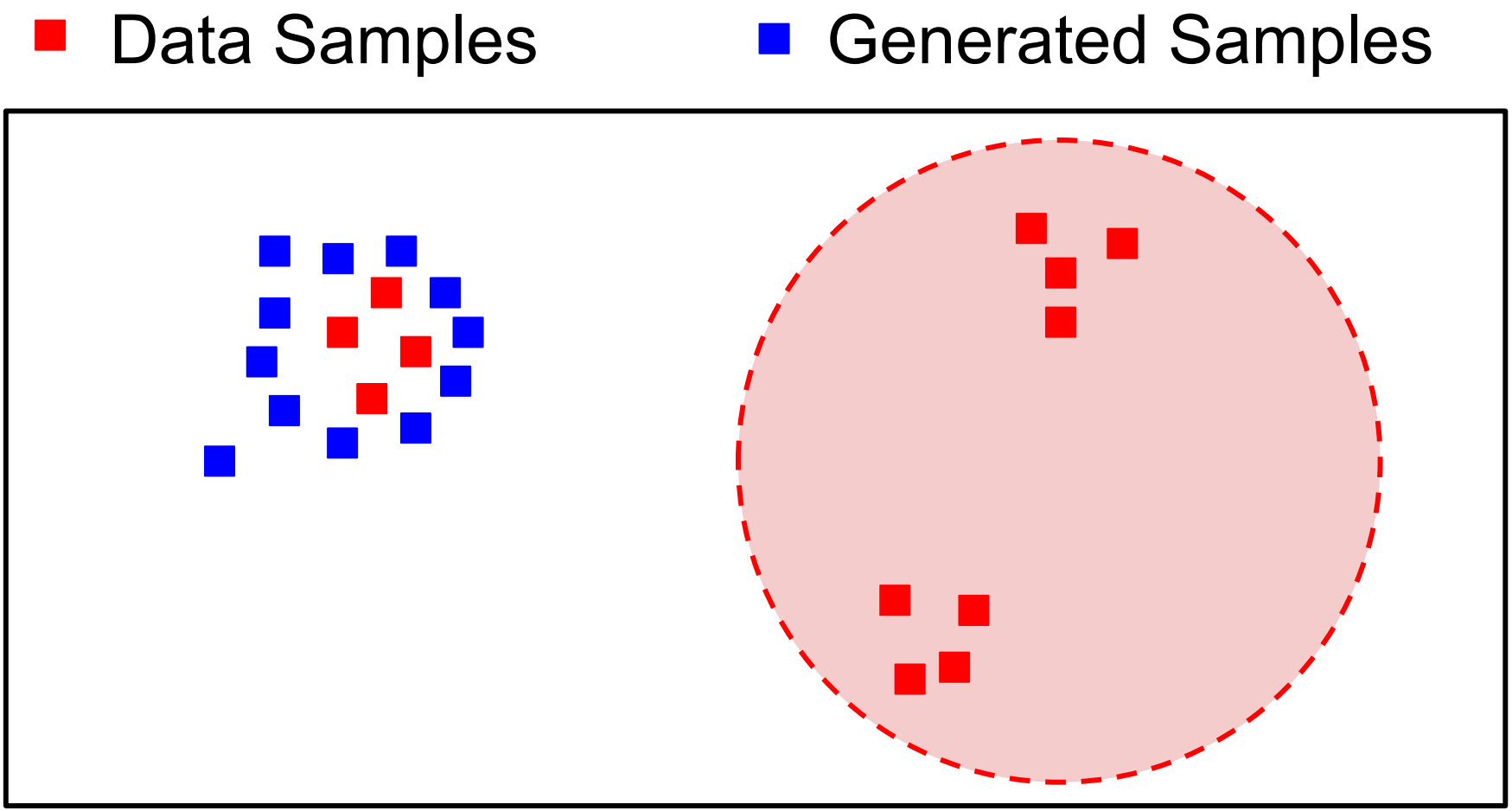


Figure: Classification regions after updating Discriminator (at this point the Discriminator tolerates misclassifying a part of real data and in return correctly classifies all the Generated samples)

Mode Collapse in GAN

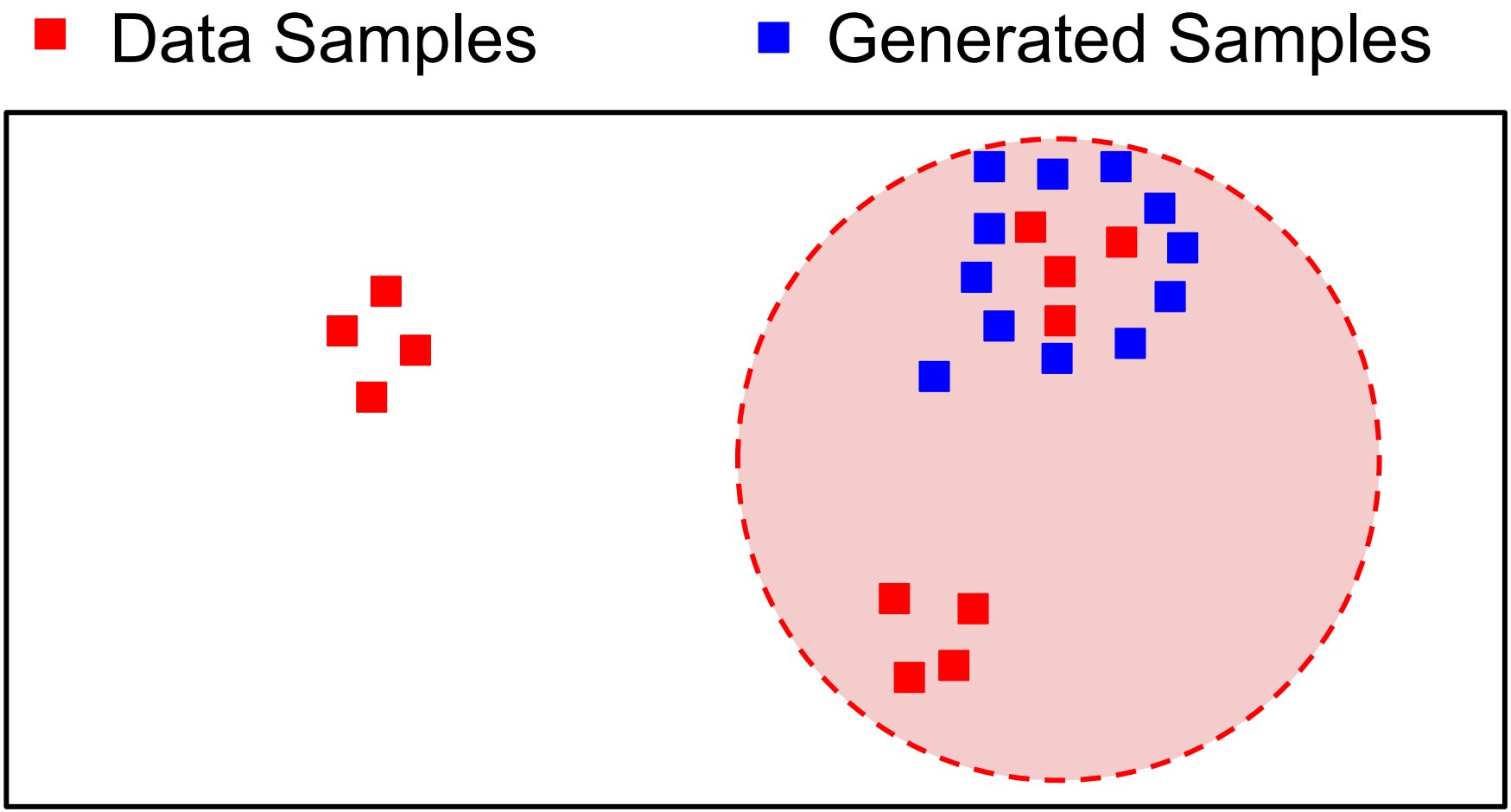


Figure: Generated samples after updating Generator (at this point the Generator concentrates the data around another data peak)

Mode Collapse in GAN

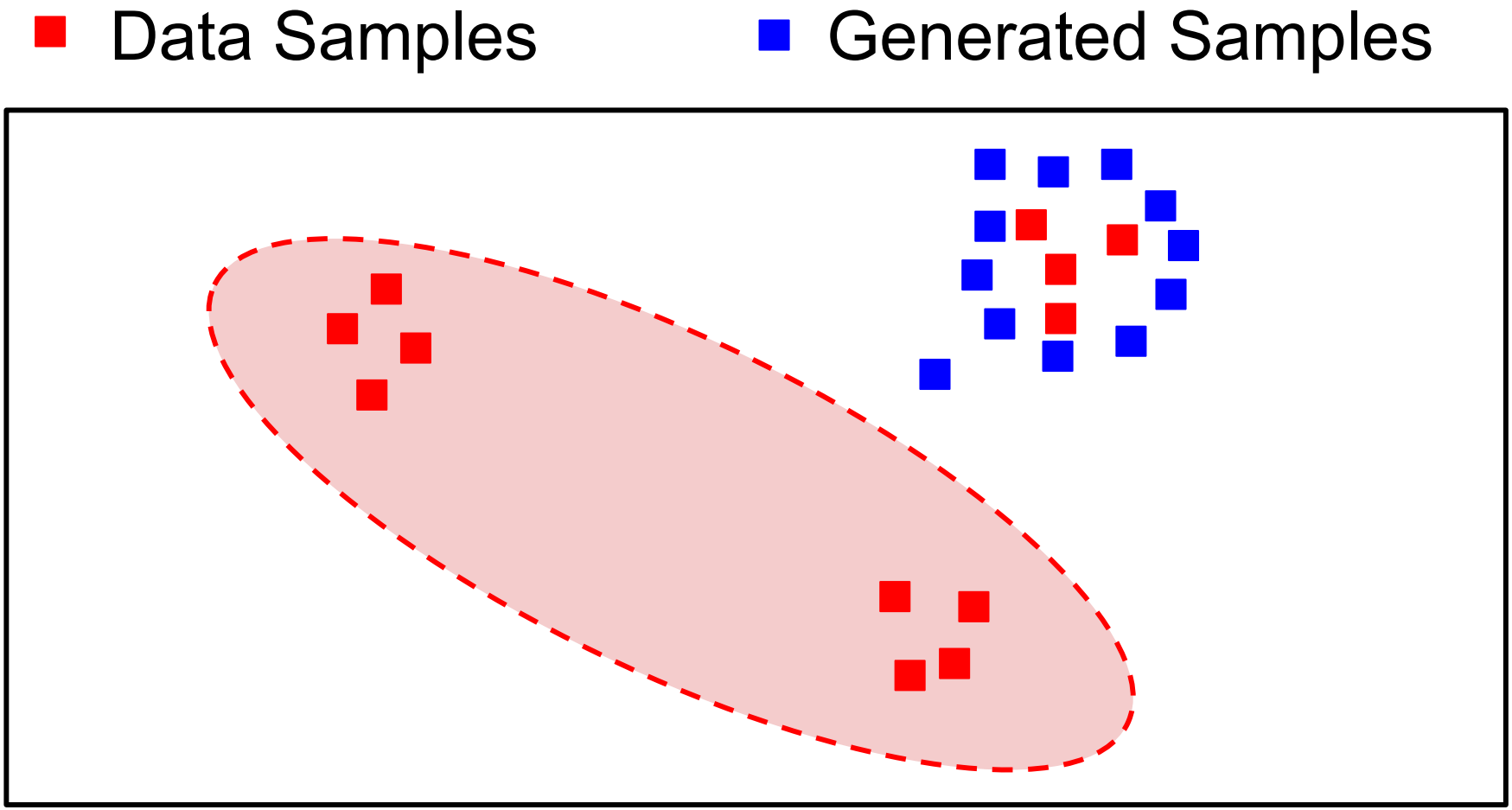


Figure: Classification regions after updating Discriminator

Mode Collapse in GAN [7]

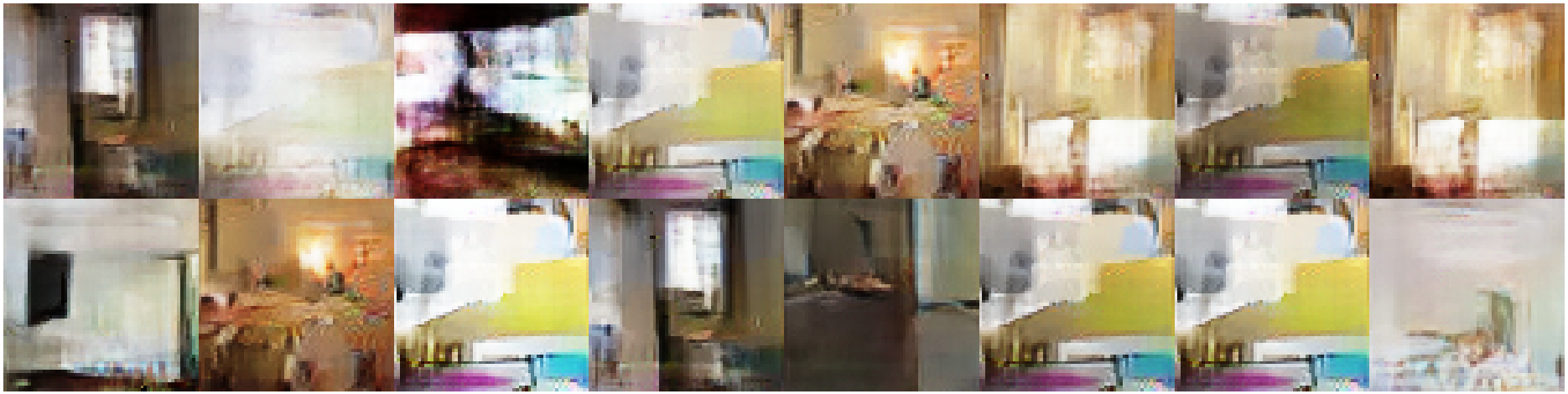


Figure: Samples form a GAN stuck in a mode (pay attention to the similar images resulting from different latent variables)

Section 7

f-GAN

From GAN Divergence to f-Divergence [8]

GAN Divergence

We have seen the original GAN uses the following divergence:

$$\mathcal{D}(p_{\text{data}}, p_\theta) = \max_\phi \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\mathbb{X})} \left[\log D_\phi(\boldsymbol{x})\right] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\mathbb{X})} \left[\log(1 - D_\phi(\boldsymbol{x}))\right]$$

f-Divergences

Given two distributions p and q over random variable $\mathbb{X}$, the f-divergence is defined as:

$$D_f(p\|q) = \int_{\boldsymbol{x}} q(\boldsymbol{x})\ f\left(\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\right) \mathrm{d}\boldsymbol{x}$$

where $f : \mathbb{R}^+ \to \mathbb{R}$ is a convex, lower-semicontinuous function satisfying $f(1) = 0$.

Variational Estimation of f-divergences [8]

Limitation

The aforementioned form of f-divergence needs the likelihood of probability while we are focused on divergences that can be measured based on samples from the distribution.

Convex Conjugate

To eliminate the above limitation, the convex conjugate definition is used. Every convex, lower-semicontinuous function f has a convex conjugate function $f^\star$, also known as Fenchel conjugate, defined as:

$$f^\star(t) = \sup_{u \in \text{dom}_f} \{ut - f(u)\}$$

The function $f^\star$ is again convex and lower-semicontinuous and the pair $(f, f^\star)$ is dual in the sense that $f^{\star\star} = f$.

Variational Estimation of f-divergences [8]

From Likelihood to Sample based f-Divergences

Using the definition of convex conjugate, now we can write:

$$f(u) = \sup_{t \in \text{dom}_{f^\star}} \{tu - f^\star(t)\}$$

We can plug the above definition to $D_f(p\|q)$ as:

$$\begin{aligned} D_f(p\|q) &= \int_{\boldsymbol{x}} q(\boldsymbol{x})\ f\left(\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\right) \mathrm{d}\boldsymbol{x} \\ &= \int_{\boldsymbol{x}} q(\boldsymbol{x}) \sup_{t \in \text{dom}_{f^\star}} \{t\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})} - f^\star(t)\}\mathrm{d}\boldsymbol{x} \end{aligned}$$

For each value of $\boldsymbol{x}$, there is a value t that results in the supremum value. Assume $\overline{T}(\boldsymbol{x})$ is that function ($\overline{T} : \mathbb{R}^D \to \text{dom}_{f^\star}$).

Variational Estimation of f-divergences [8]

From Likelihood to Sample based f-Divergences (Cont.)

Using $\overline{T}(\boldsymbol{x})$ definition, we have:

$$D_f(p\|q) = \int_{\boldsymbol{x}} q(\boldsymbol{x}) \left(\overline{T}(\boldsymbol{x}) \frac{p(\boldsymbol{x})}{q(\boldsymbol{x})} - f^\star(\overline{T}(\boldsymbol{x})) \right) \mathrm{d}\boldsymbol{x}$$

Now assume two cases:

- If you access all possible functions, then you select $\overline{T}(\boldsymbol{x})$ which leads to the above equality.
- Limit your options to a family of functions, namely $\mathcal{T}$. Then we can find the best approximation to $\overline{T}(\boldsymbol{x})$ as $T(\boldsymbol{x})$. So, instead of f-Divergence, you can find its lower bound (thus you have a variational estimation).

Thus:

$$D_f(p\|q) \geq \sup_{T \in \mathcal{T}} \int_{\boldsymbol{x}} q(\boldsymbol{x}) \left(T(\boldsymbol{x}) \frac{p(\boldsymbol{x})}{q(\boldsymbol{x})} - f^\star(T(\boldsymbol{x})) \right) \mathrm{d}\boldsymbol{x}$$

Variational Estimation of f-divergences [8]

From Likelihood to Sample based f-Divergences (Cont.)

Now let's simplify our lower-bound:

$$
\begin{aligned}
D_f(p\|q) &\geq \sup_{T\in\mathcal{T}} \int_{\boldsymbol{x}} q(\boldsymbol{x}) \left(T(\boldsymbol{x})\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})} - f^\star(T(\boldsymbol{x})) \right) \mathrm{d}\boldsymbol{x} \\
&= \sup_{T\in\mathcal{T}} \left(\int_{\boldsymbol{x}} p(\boldsymbol{x}) T(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} - \int_{\boldsymbol{x}} q(\boldsymbol{x}) f^\star(T(\boldsymbol{x})) \mathrm{d}\boldsymbol{x} \right) \\
&= \sup_{T\in\mathcal{T}} \left(\mathbb{E}_{\boldsymbol{x}\sim p(\boldsymbol{x})} \left[T(\boldsymbol{x}) \right] - \mathbb{E}_{x\sim q(\boldsymbol{x})} \left[f^\star(T(\boldsymbol{x})) \right] \right)
\end{aligned}
$$

Thus:

- You have again a variational lower bound estimation on the f-Divergences.
- Your estimation is likelihood-free as we need.
- Assume $p = p_{\text{data}}$ and $q = p_\theta$, then you can minimize the above variational lower bound to train the generator.

☞ But, how to select $\mathcal{T}$ and what is $f^\star$?

Selecting $\mathcal{T}$

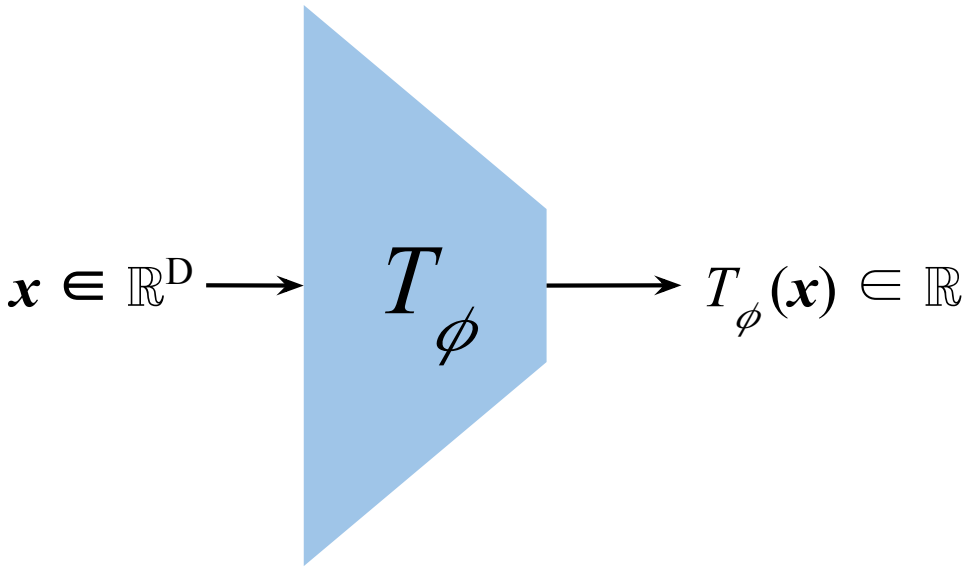


Figure: $\mathcal{T}$ implemented using Neural Networks ($g_f : \mathbb{R} \to \text{dom}_{f^\star}$ is used to respect the convex conjugate definition $f(u) = \sup_{t \in \text{dom}_{f^\star}} \{tu - f^\star(t)\}$)

Selecting $\mathcal{T}$

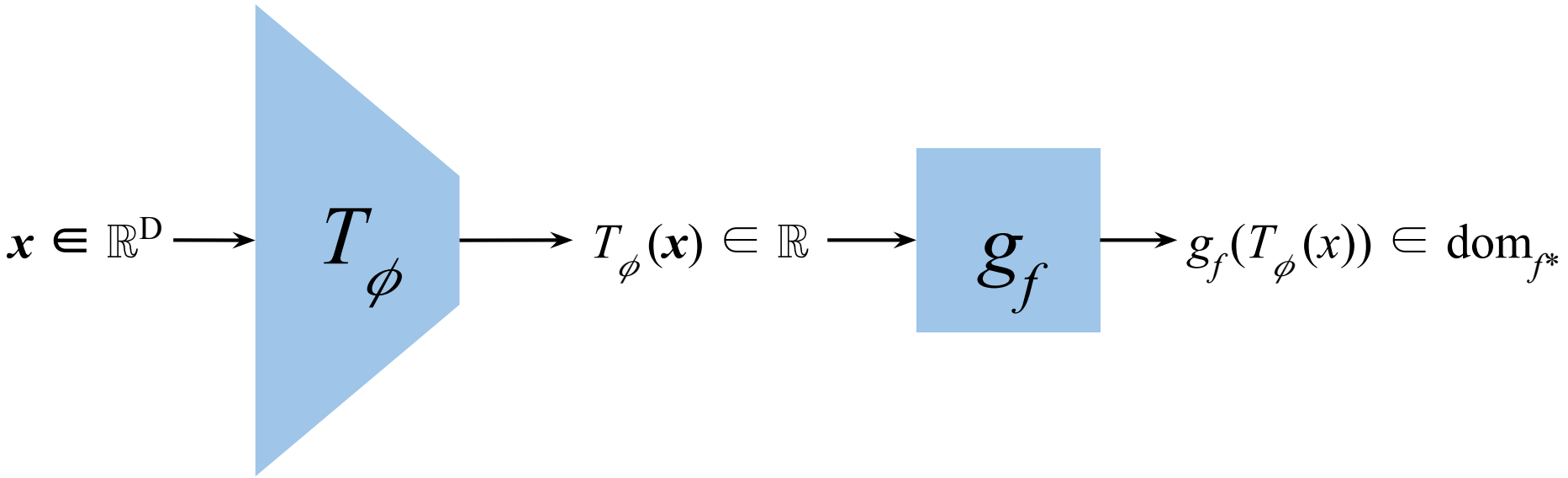


Figure: $\mathcal{T}$ implemented using Neural Networks ($g_f : \mathbb{R} \to \text{dom}_{f^\star}$ is used to respect the convex conjugate definition $f(u) = \sup_{t \in \text{dom}_{f^\star}} \{tu - f^\star(t)\}$)

Selecting f

Table: Two selection for f-divergences (for a more complete list, refer to [8])

name	KL	GAN
f	$u \log u$	$u \log u - (u+1)\log(u+1)$
$f^\star$	$\exp(t-1)$	$-\log(1-\exp(t))$
$\text{dom}_{f^\star}$	$\mathbb{R}$	$\mathbb{R}^-$
g_f	ν	$-\log(1+\exp(-\nu))$

f-GAN and connection to GAN

f-GAN

The f-GAN final objective function is:

$$\min_{\theta} \max_{\phi} F(\boldsymbol{\theta}, \boldsymbol{\phi})$$

where we define:

$$F(\boldsymbol{\theta}, \boldsymbol{\phi}) \triangleq \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[g_f \left(T_{\phi}(\boldsymbol{x}) \right) \right] + \mathbb{E}_{\boldsymbol{x} \sim p_{\theta}(\mathbb{X})} \left[-f^{\star} \left(g_f \left(T_{\phi}(\boldsymbol{x}) \right) \right) \right]$$

GAN as Special f-GAN

Using the definitions of f, $f^{\star}$ and g_f in Slide 59, if we define:

$$D_{\phi}(\boldsymbol{x}) = g_f(T_{\phi}(\boldsymbol{x})) = \frac{1}{1 + \exp(-T(\boldsymbol{x}))}$$

Then we can easily show the objective function of f-GAN can be rewritten as:

$$F(\theta, \boldsymbol{\phi}) = \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\log D_{\phi}(\boldsymbol{x}) \right] + \mathbb{E}_{\boldsymbol{x} \sim p_{\theta}(\mathbb{X})} \left[\log \left(1 - D_{\phi}(\boldsymbol{x}) \right) \right]$$

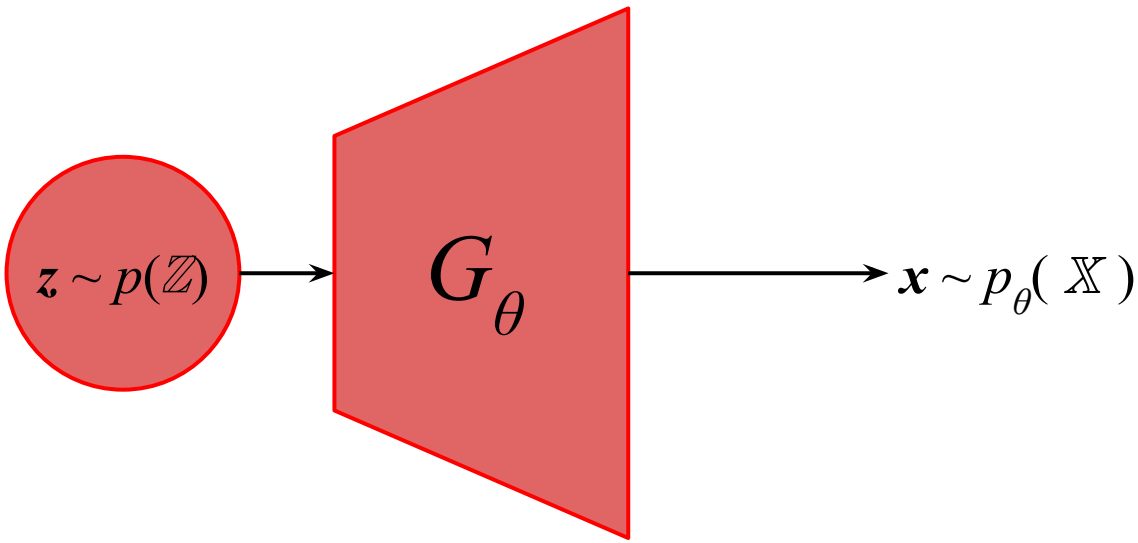


Figure: f-GAN architecture ($G_\theta : \mathbb{R}^m \to \mathbb{R}^D$, $T_\phi : \mathbb{R}^D \to \mathbb{R}$ and $g_f : \mathbb{R} \to \text{dom}_{f^\star}$)

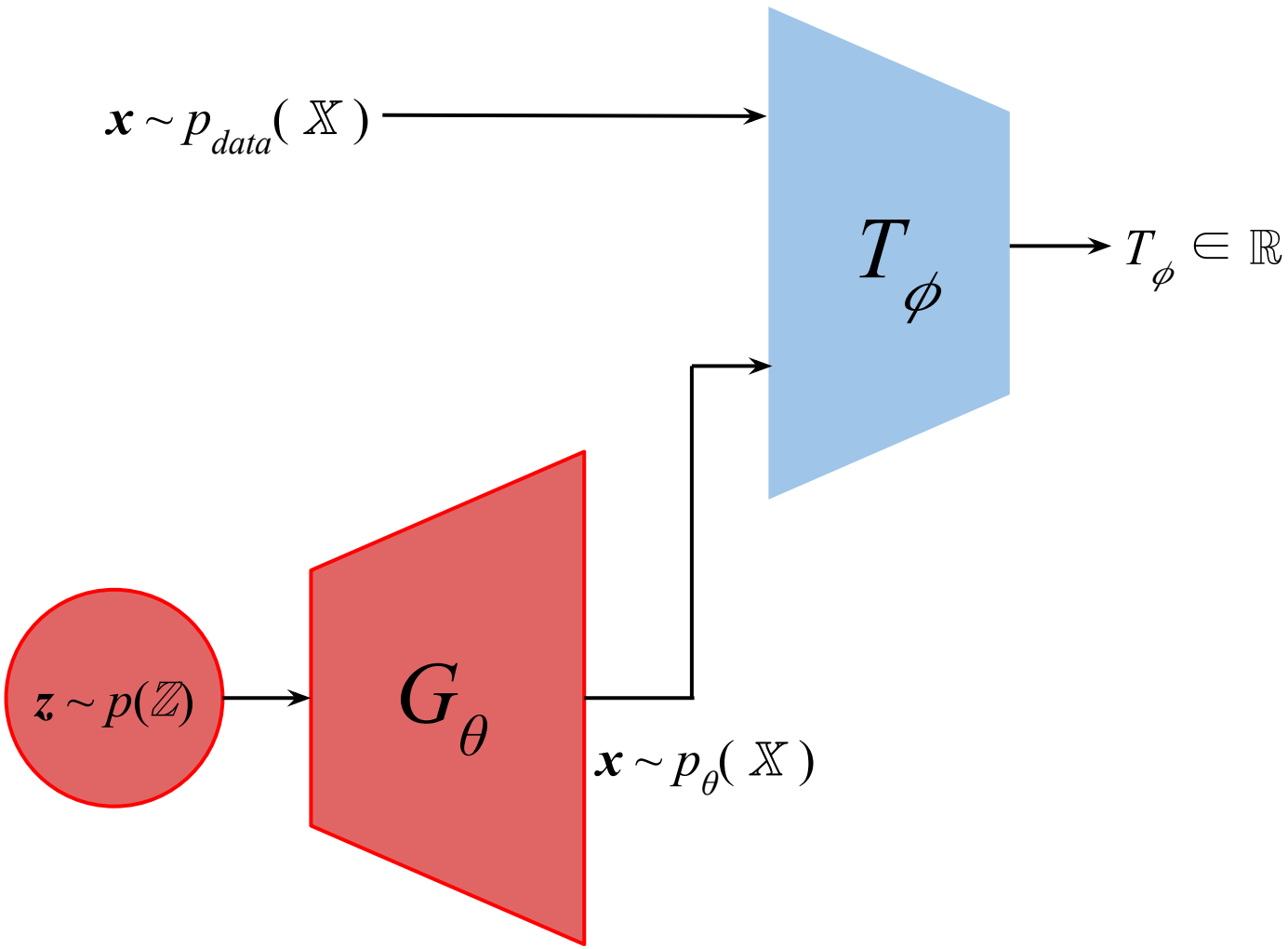


Figure: f-GAN architecture ($G_\theta : \mathbb{R}^m \to \mathbb{R}^D$, $T_\phi : \mathbb{R}^D \to \mathbb{R}$ and $g_f : \mathbb{R} \to \text{dom}_{f^\star}$)

f-GAN

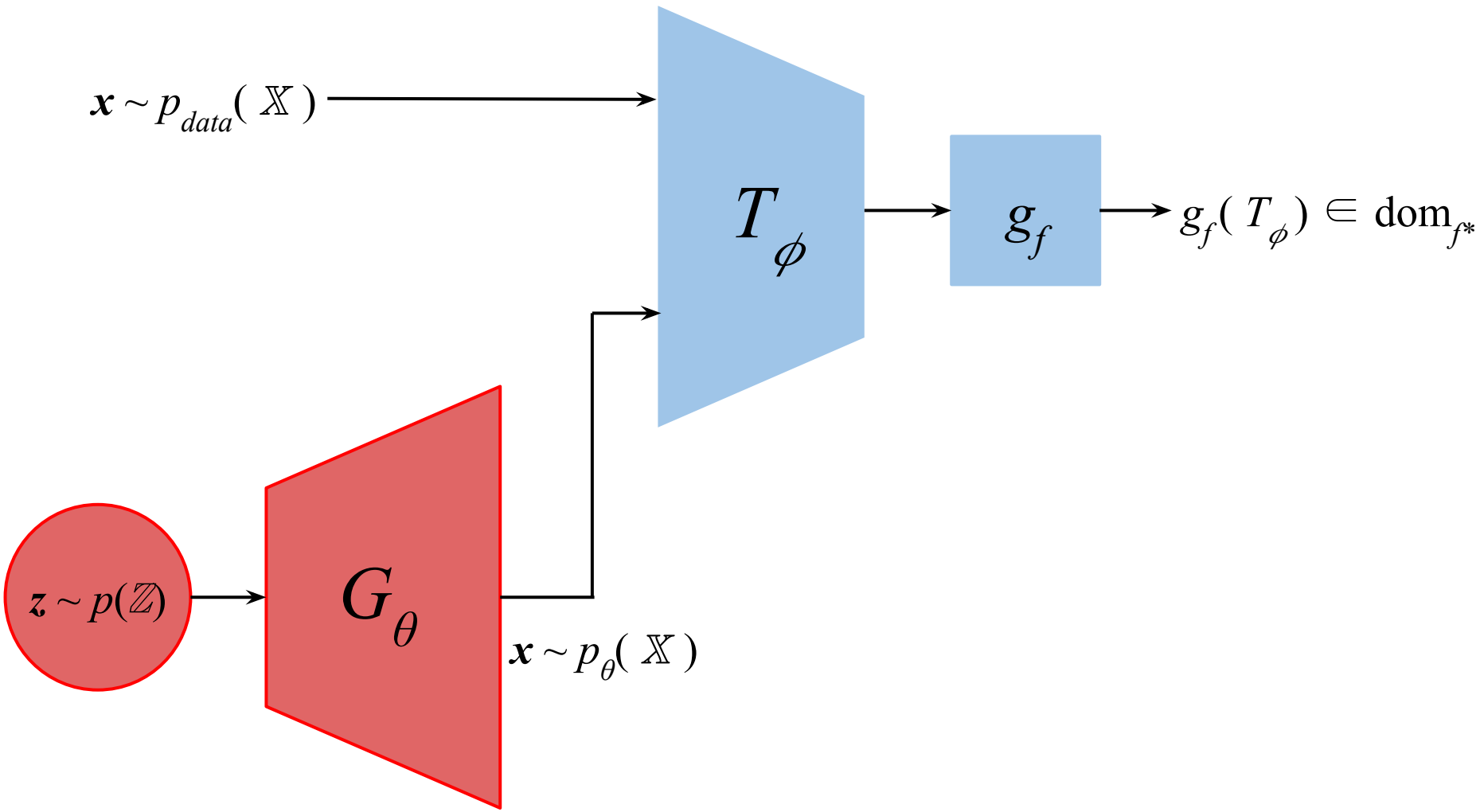
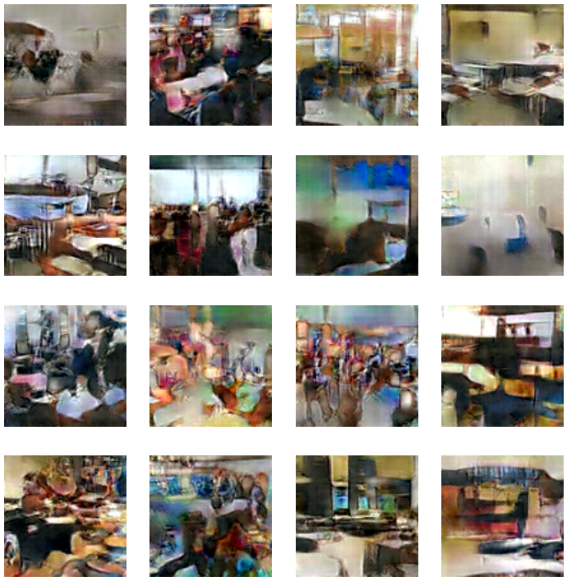
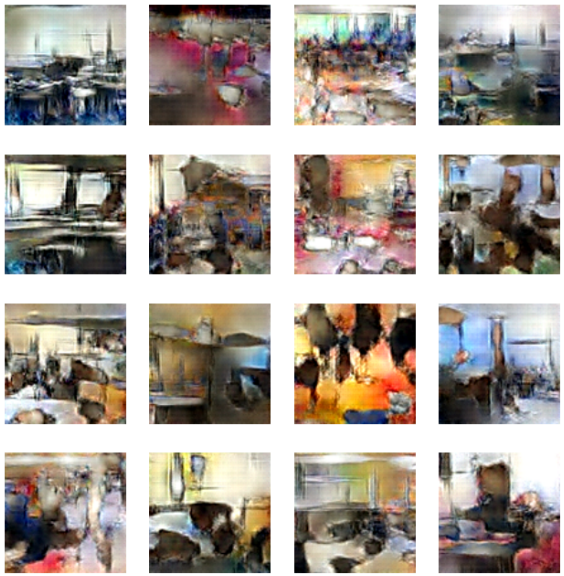


Figure: f-GAN architecture ($G_\theta : \mathbb{R}^m \to \mathbb{R}^D$, $T_\phi : \mathbb{R}^D \to \mathbb{R}$ and $g_f : \mathbb{R} \to \text{dom}_{f^\star}$)

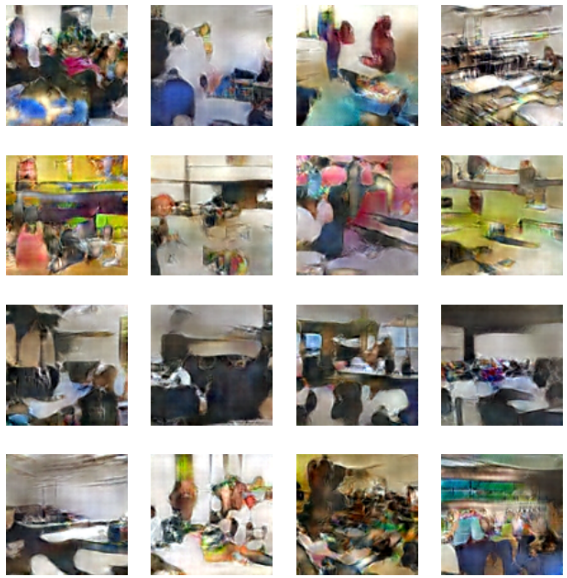
f-GAN Samples [8]



(a) GAN



(b) KL



(c) Squared Hellinger

Figure: f-GAN results over LSUN dataset using different f-divergences

Section 8

Bidirectional Generative Adversarial Networks

Limitation with GAN

Generative Model Tasks

Based on GAN formulation we have:

- ✗ Density estimation $p(\boldsymbol{x}_{\text{new}})$: GANs are designed to be likelihood-free
- ✓ Generation: $\boldsymbol{x}_{\text{new}} \sim p(\boldsymbol{x})$: GANs are designed for efficient sampling
- ✗ Latent representation $\boldsymbol{z}$: High level representation can be greatly useful for unsupervised learning
  - ✓ Provide latent representation for their generated samples
  - ✗ Cannot provide latent representation for dataset samples

Bidirectional Generative Adversarial Networks [9]

BiGANs modify the original GAN such that it can provide latent representation for any sample.

BiGAN Architecture [9]

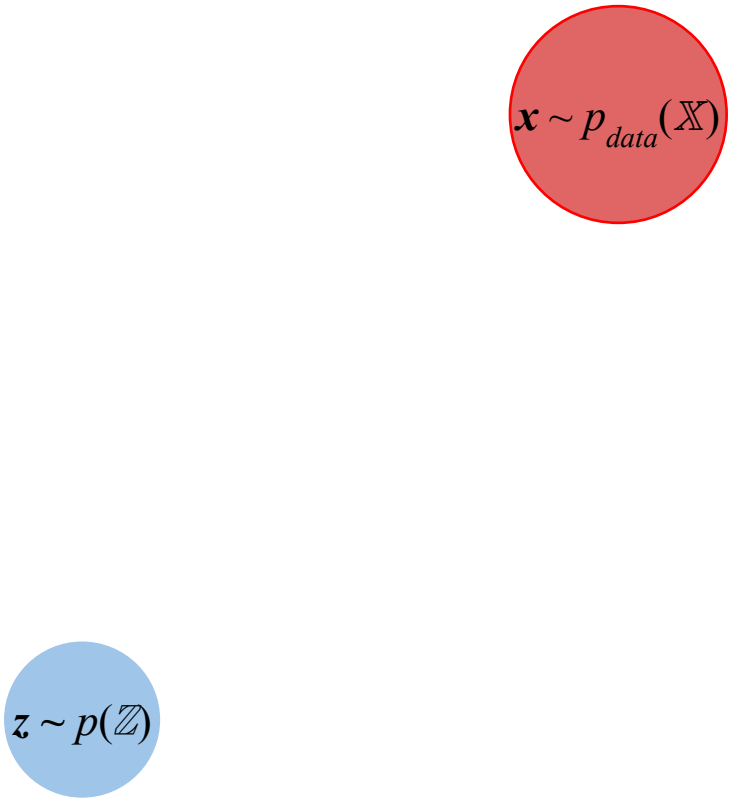


Figure: BiGAN architecture

BiGAN Architecture [9]

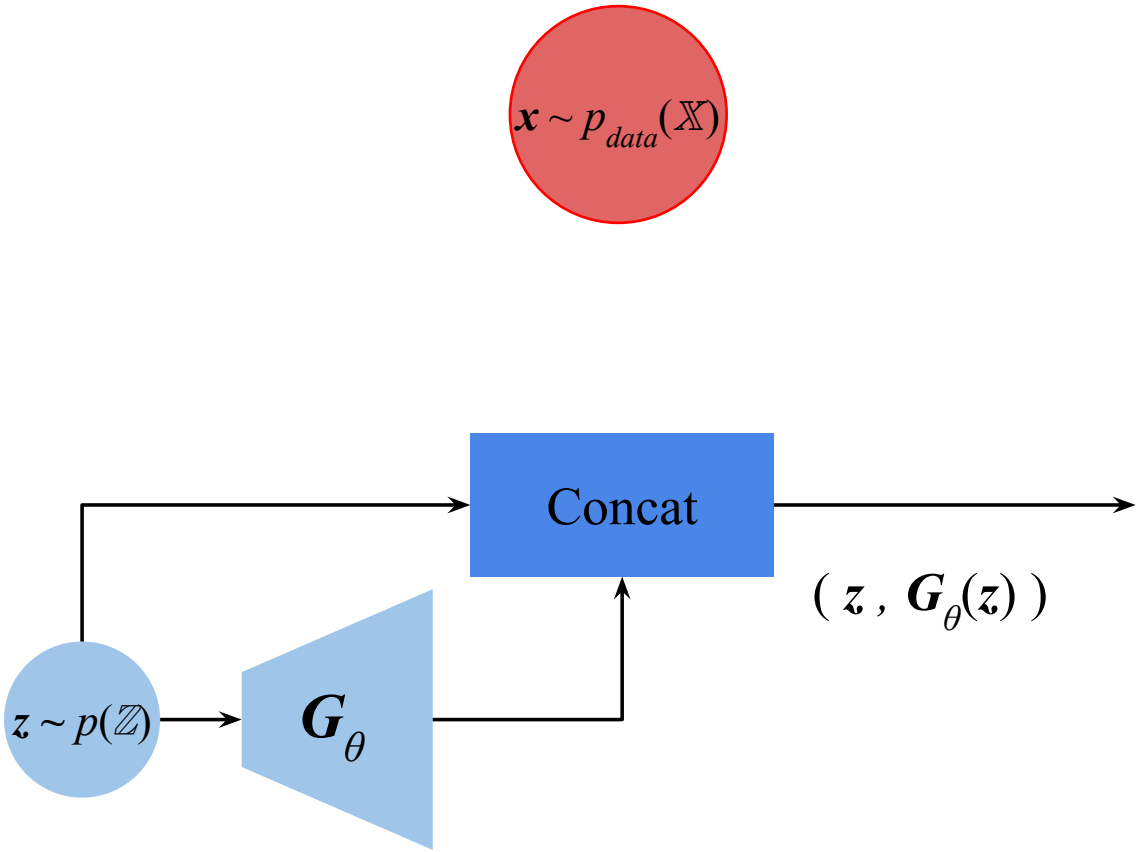


Figure: BiGAN architecture

BiGAN Architecture [9]

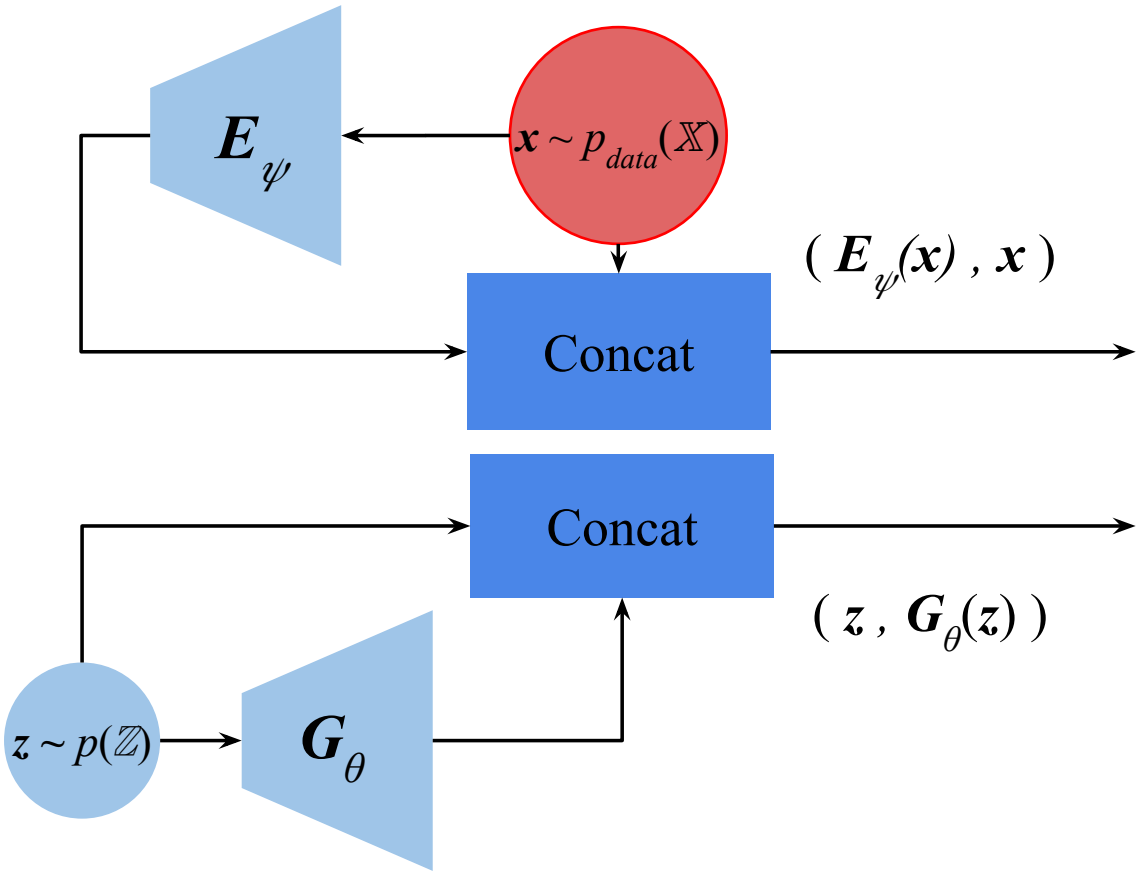


Figure: BiGAN architecture

BiGAN Architecture [9]

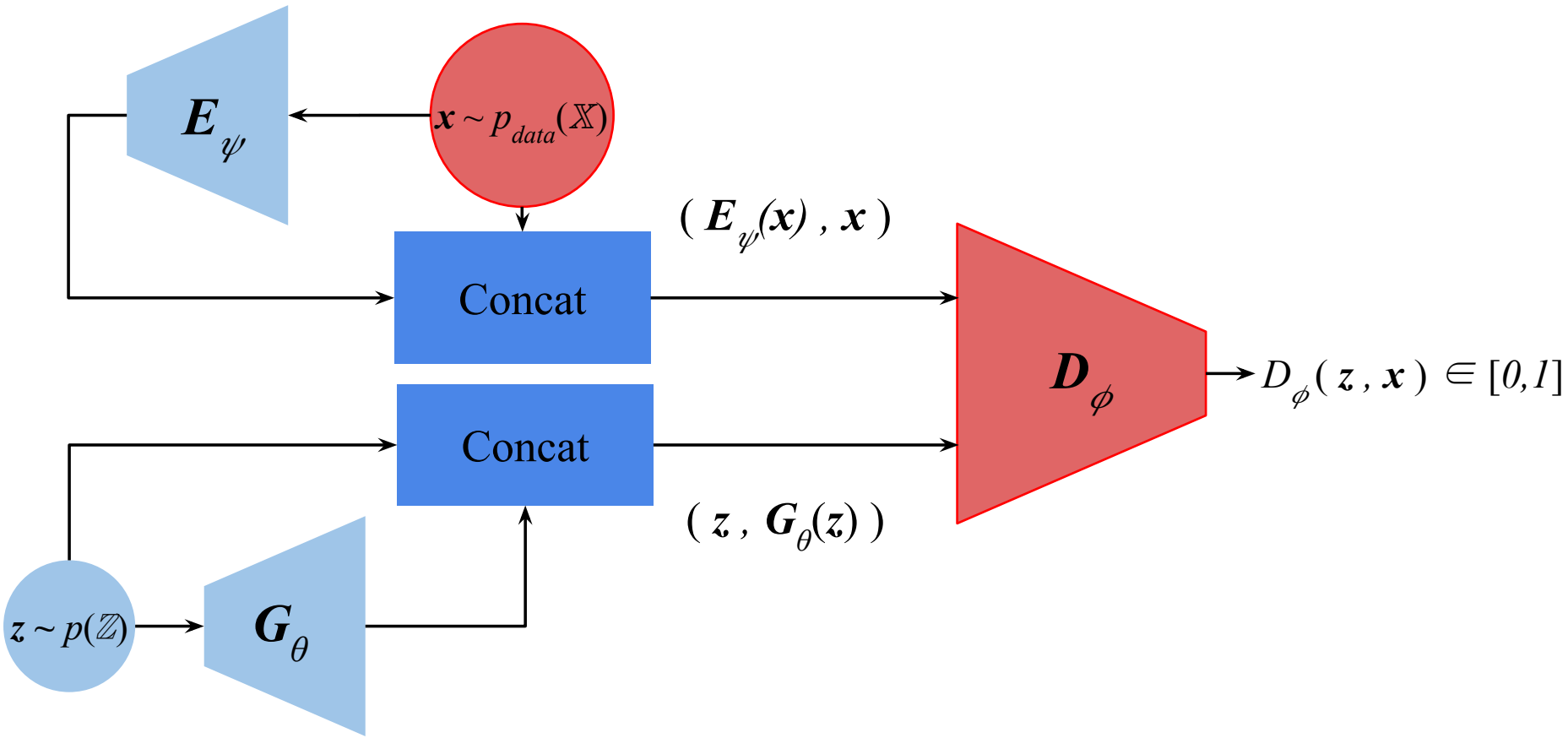


Figure: BiGAN architecture

BiGAN Architecture (Source: [9])

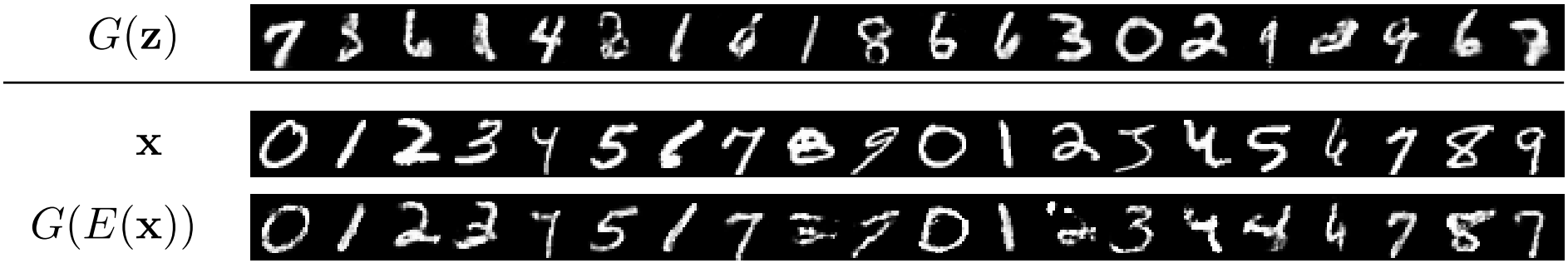


Figure: BiGAN results over MNIST dataset

BiGAN Architecture [9]

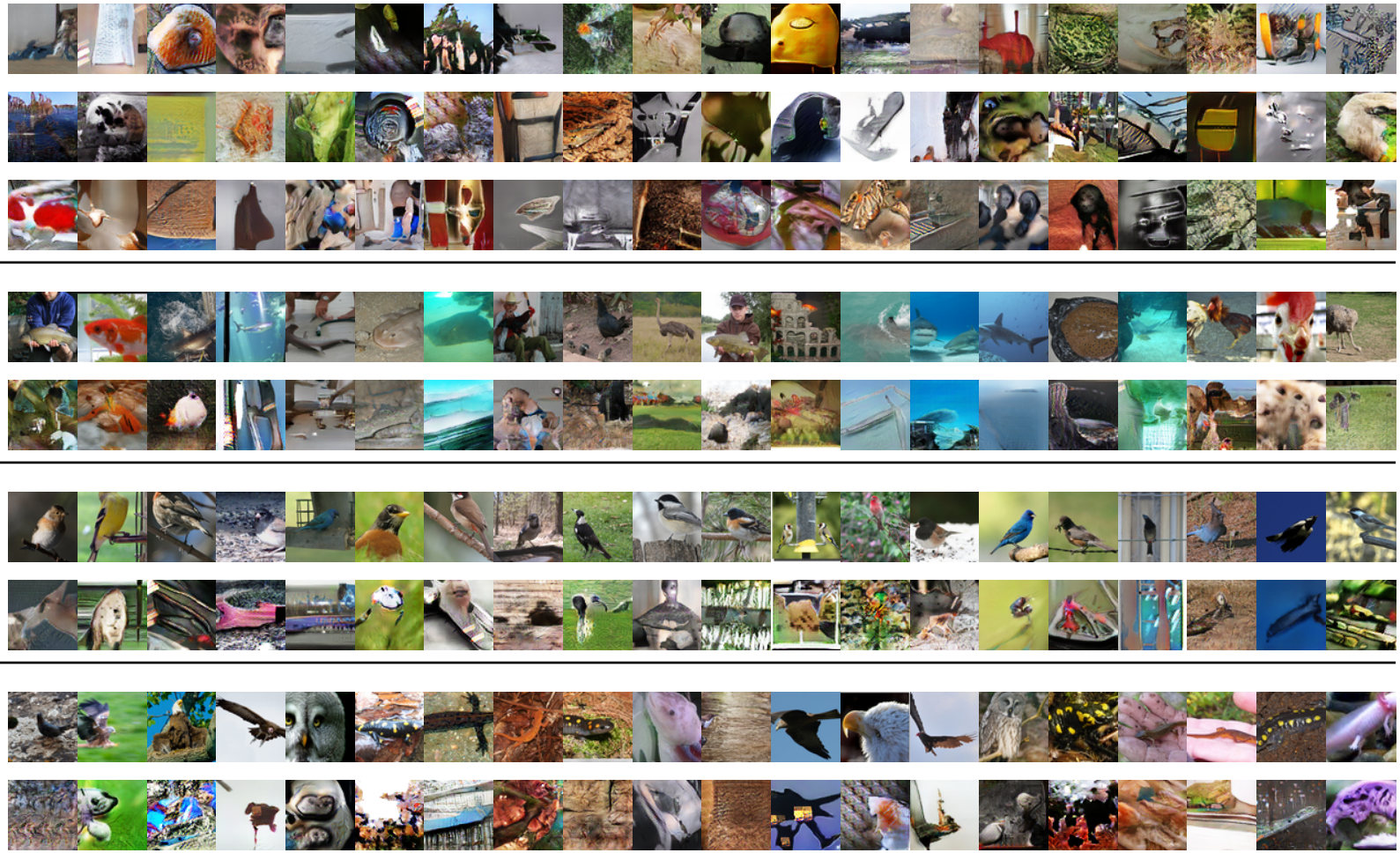


Figure: BiGAN results over ImageNet dataset

Section 9

CycleGAN

Image-to-Image Translation - Paired Dataset

Figure: Sample of paired dataset for image-to-image translation (source: [10])

Image-to-Image Translation - Paired

Assume you have access to a paired dataset of images as $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^N$. Examples of such datasets can be:

- $\boldsymbol{x}_i$: Object sketch, $\boldsymbol{y}_i$: Object image
- $\boldsymbol{x}_i$: grayscale image, $\boldsymbol{y}_i$: Color image

Your objective is to find two mappings as follows:

- $G : \mathcal{X} \rightarrow \mathcal{Y}$ such that $\boldsymbol{y}_i = G(\boldsymbol{x}_i)$
- $F : \mathcal{Y} \rightarrow \mathcal{X}$ such that $\boldsymbol{x}_i = F(\boldsymbol{y}_i)$

Several works can be used in this supervised setting [11, 12, 13].

Image-to-Image Translation - Unpaired Dataset

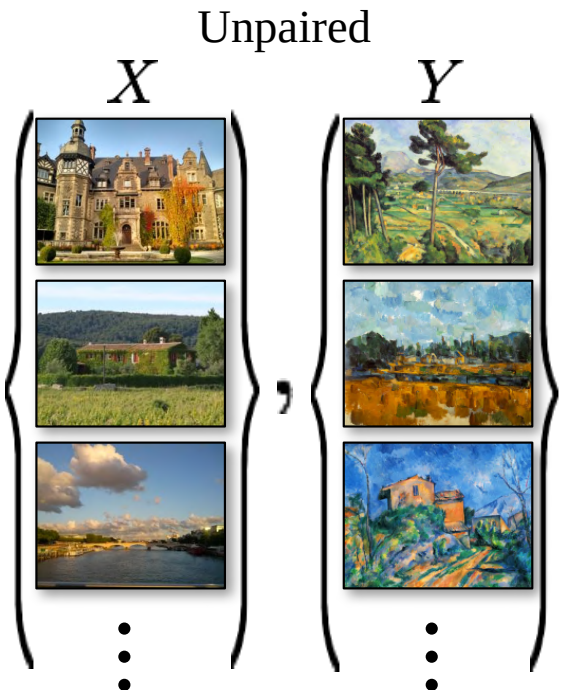


Figure: Sample of unpaired dataset for image-to-image translation (source: [10])

CycleGAN Problem Statement [10]

The CycleGAN (primarily known as Cycle-Consistent Adversarial Networks) problem is Image-to-Image Translation using ***unpaired*** dataset.
Assume you have access to unpaired dataset of images as $\{\{\boldsymbol{x}_i\}_{i=1}^N, \{\boldsymbol{y}_i\}_{i=1}^N\}$. In this case, we can assume two data distributions:

$$\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X}), \boldsymbol{y} \sim p_{\text{data}}(\mathbb{Y})$$

Your objective is to find two mappings as follows:

- $G : \mathcal{X} \rightarrow \mathcal{Y}$ such that $\{G(\boldsymbol{x}_i)\}_{i=1}^N$ resembles $p_{\text{data}}(\mathbb{Y})$ samples such as $\{\boldsymbol{y}_i\}_{i=1}^N$
- $F : \mathcal{Y} \rightarrow \mathcal{X}$ such that $\{F(\boldsymbol{y}_i)\}_{i=1}^N$ resembles $p_{\text{data}}(\mathbb{X})$ samples such as $\{\boldsymbol{x}_i\}_{i=1}^N$

CycleGAN [10]

Mapping G Design

We want mapping $G : \mathcal{X} \to \mathcal{Y}$ such that $\{G(\boldsymbol{x}_i)\}_{i=1}^N$ resembles $p_{\text{data}}(\mathbb{Y})$ samples:

- ☞ Equivalently, Discriminator D_Y cannot discriminate $\{G(\boldsymbol{x}_i)\}_{i=1}^N$ and $\{\boldsymbol{y}_i\}_{i=1}^N$. The objective function is:

$$\mathcal{L}_{\text{GAN}}(G, D_Y, \mathcal{X}, \mathcal{Y}) = \mathbb{E}_{\boldsymbol{y} \sim p_{\text{data}}(\mathbb{Y})} \left[\log D_Y(\boldsymbol{y})\right] + \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\log \left(1 - D_Y(G(\boldsymbol{x}))\right)\right]$$

Mapping F Design

The problem of designing F mapping can also be handled by the introduction of Discriminator D_X and using the following GAN-based objective function:

$$\mathcal{L}_{\text{GAN}}(F, D_X, \mathcal{Y}, \mathcal{X}) = \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\log D_X(\boldsymbol{x})\right] + \mathbb{E}_{\boldsymbol{y} \sim p_{\text{data}}(\mathbb{Y})} \left[\log \left(1 - D_X(F(\boldsymbol{y}))\right)\right]$$

Cycle Consistency [10]

Challenge

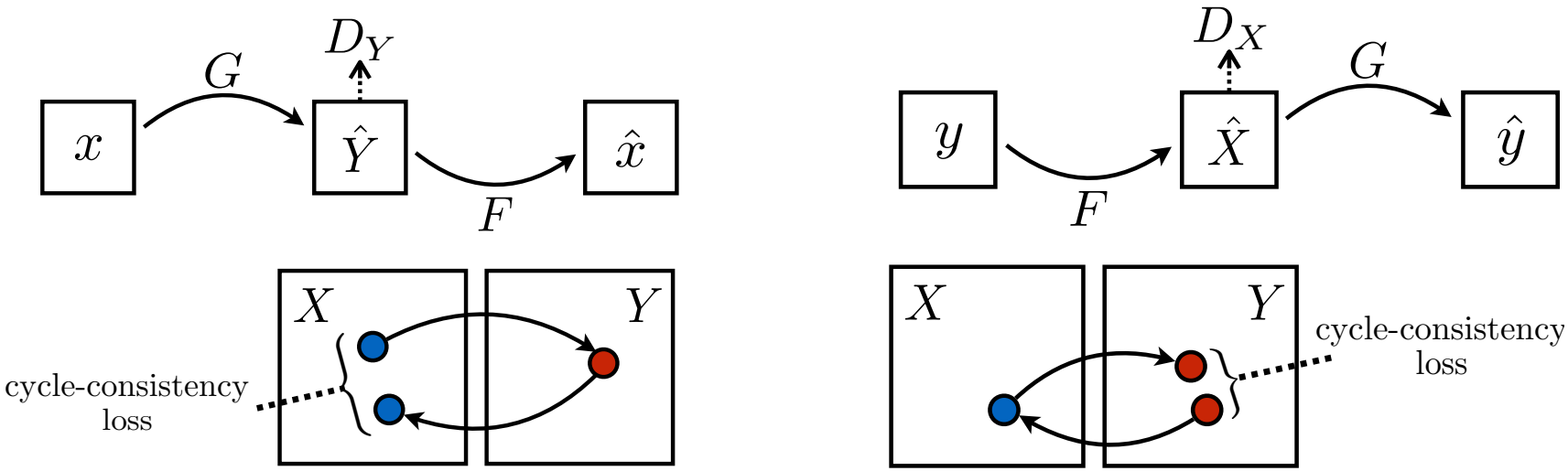
Assume we train F and G using the following objective function:

$$\min_{F,G} \mathcal{L}_{\text{GAN}}(G, D_Y, \mathcal{X}, \mathcal{Y}) + \mathcal{L}_{\text{GAN}}(F, D_X, \mathcal{Y}, \mathcal{X})$$

Then the following scenario may happen due to the high capacity of mappings:

- ✗ G generate sample $G(\boldsymbol{x})$ similar to $\{\boldsymbol{y}_i\}_{i=1}^N$ irrespective of $\boldsymbol{x}$.
- ✗ F generate sample $F(\boldsymbol{y})$ similar to $\{\boldsymbol{x}_i\}_{i=1}^N$ irrespective of $\boldsymbol{y}$.
- ☞ The above challenge can be solved by adding cycle consistency loss to the above objective function.

Cycle Consistency Loss [10]



(a) Forward cycle consistency (source: [10])

(b) Backward cycle consistency (source: [10])

CycleGAN Objective

The CycleGAN loss function is:

$$\mathcal{L}_{\text{cyc}}(G, F) = \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\mathbb{X})} \left[\|F(G(\boldsymbol{x})) - \boldsymbol{x}\|_1 \right] + \mathbb{E}_{\boldsymbol{y} \sim p_{\text{data}}(\mathbb{Y})} \left[\|G(F(\boldsymbol{y})) - \boldsymbol{y}\|_1 \right]$$

and the Final CycleGAN objective function is:

$$\min_{F,G} \mathcal{L}_{\text{GAN}}(G, D_Y, \mathcal{X}, \mathcal{Y}) + \mathcal{L}_{\text{GAN}}(F, D_X, \mathcal{Y}, \mathcal{X}) + \lambda \mathcal{L}_{\text{cyc}}(G, F)$$

CycleGAN Results

Figure: CycleGAN results for Edges$leftrightarrow$Shoes transformation (source: [10])

CycleGAN Results

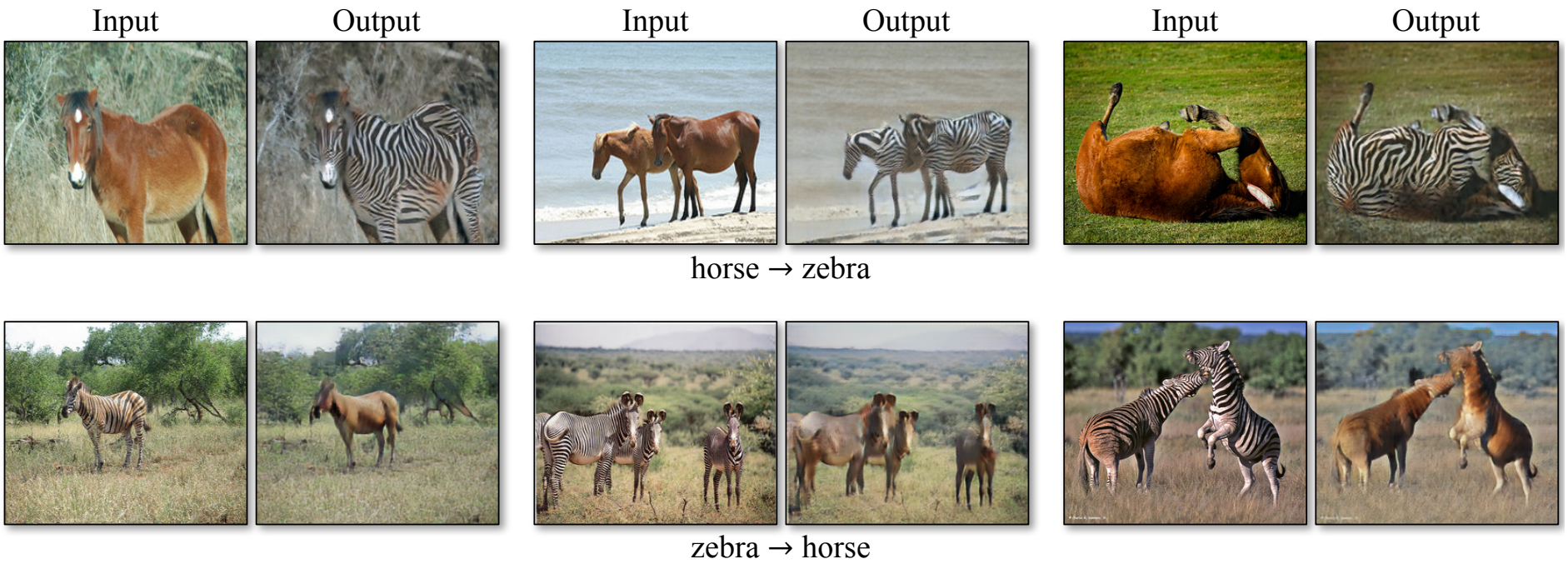


Figure: CycleGAN results for Zebra↔Horse transformation (source: [10])

CycleGAN Results

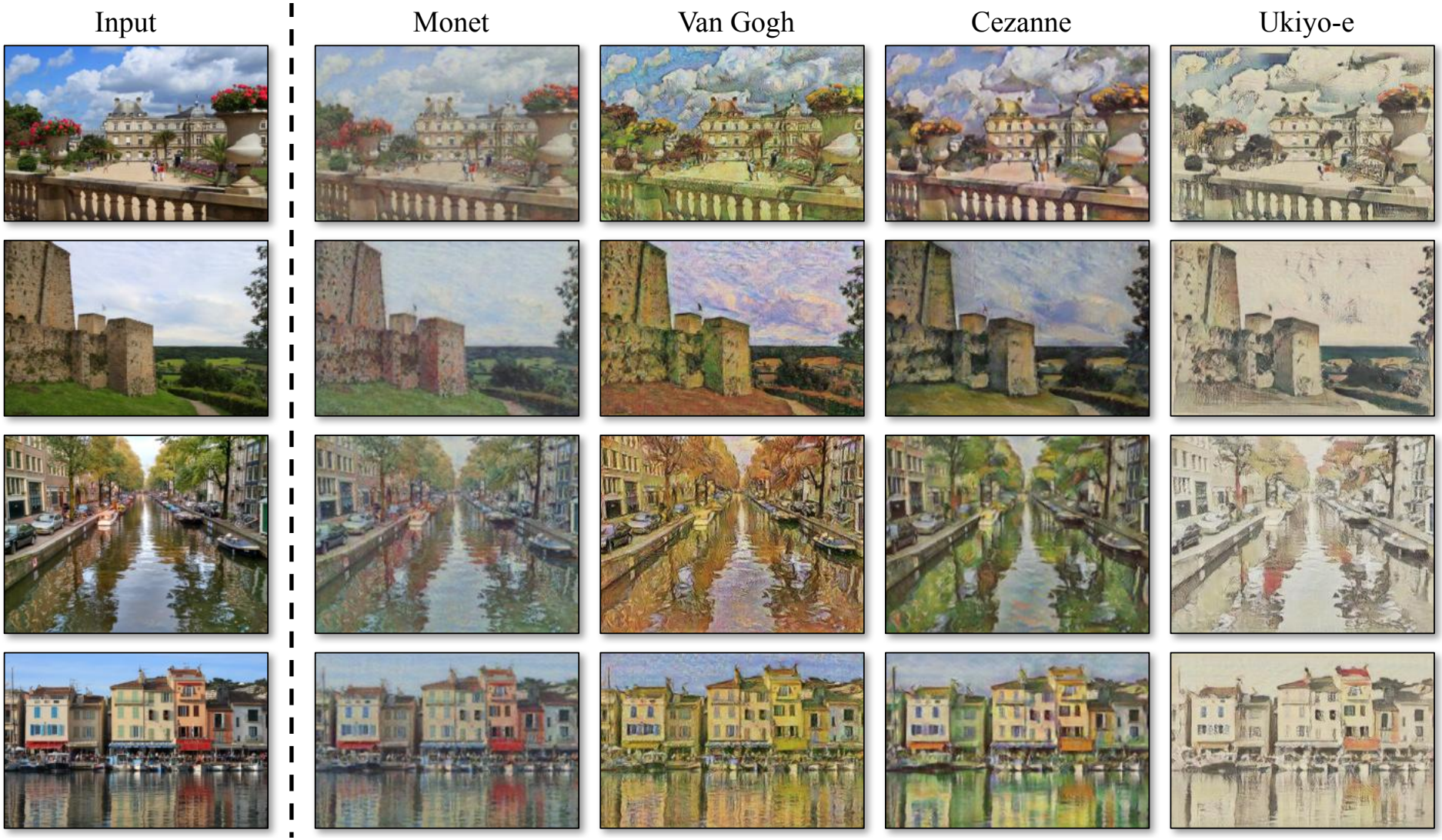


Figure: CycleGAN results for painting style transformation (source: [10])

Section 10

Conclusion

Conclusion

Generative Adversarial Nets

- Pros
  - High-Quality Generation
  - The freedom in choosing a Generator model
  - Different innovative ideas upon GAN loss (such as CycleGAN)
- Cons
  - No likelihood calculation
  - Hard training especially due to mode Collapse
  - Hard to monitor during training
  - No latent inference in the original setting

List of Abbreviations

Complete	Abbreviation
Generative Adversarial Net	GAN
Monte Carlo Estimation	MCE

References I

Lucas Theis, Aäron van den Oord, and Matthias Bethge,
"A note on the evaluation of generative models,"
arXiv preprint arXiv:1511.01844, 2015.

Aäron van den Oord and Joni Dambre,
"Locally-connected transformations for deep gmms,"
in *International Conference on Machine Learning (ICML): Deep learning Workshop*, 2015, pp. 1–8.

Kevin P Murphy,
Probabilistic machine learning: an introduction,
MIT press, 2022.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio,
"Generative adversarial nets,"
Advances in neural information processing systems, vol. 27, 2014.

Alec Radford, Luke Metz, and Soumith Chintala,
"Unsupervised representation learning with deep convolutional generative adversarial networks,"
arXiv preprint arXiv:1511.06434, 2015.

References II

Guangyuan Zhang, Xiaoping Rui, Stefan Poslad, Xianfeng Song, Yonglei Fan, and Bang Wu,
"A method for the estimation of finely-grained temporal spatial human population density distributions based on cell phone call detail records,"
Remote Sensing, vol. 12, no. 16, pp. 2572, 2020.

Martin Arjovsky, Soumith Chintala, and Léon Bottou,
"Wasserstein gan: Machine learning,"
stat. ML, 2017.

Sebastian Nowozin, Botond Cseke, and Ryota Tomioka,
"f-gan: Training generative neural samplers using variational divergence minimization,"
Advances in neural information processing systems, vol. 29, 2016.

Jeff Donahue, Philipp Krähenbühl, and Trevor Darrell,
"Adversarial feature learning,"
arXiv preprint arXiv:1605.09782, 2016.

Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A Efros,
"Unpaired image-to-image translation using cycle-consistent adversarial networks,"
in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2223–2232.

David Eigen and Rob Fergus,
"Predicting depth, surface normals and semantic labels with a common multi-scale convolutional architecture,"
in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 2650–2658.

References III

Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros,
"Image-to-image translation with conditional adversarial networks,"
in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 1125–1134.

Richard Zhang, Phillip Isola, and Alexei A Efros,
"Colorful image colorization,"
in *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part III 14*. Springer, 2016, pp. 649–666.